General-purpose Timers

CSE 4095
Special Topics in Computer Engineering III:
Introduction to Embedded Systems
Spring 2023

Taken from Dr. Yifeng Zhu's Lecture Notes
Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C – Chapter 15

Clocks and Timers

- Clock
  - Microcontrollers usually have many different clock sources that have different
    - frequency (relates to precision)
    - energy consumption
    - stability, e.g., crystal-controlled clock vs. digitally controlled oscillator

- Timer
  - Counting events
  - Creating delays
  - Measuring time between events

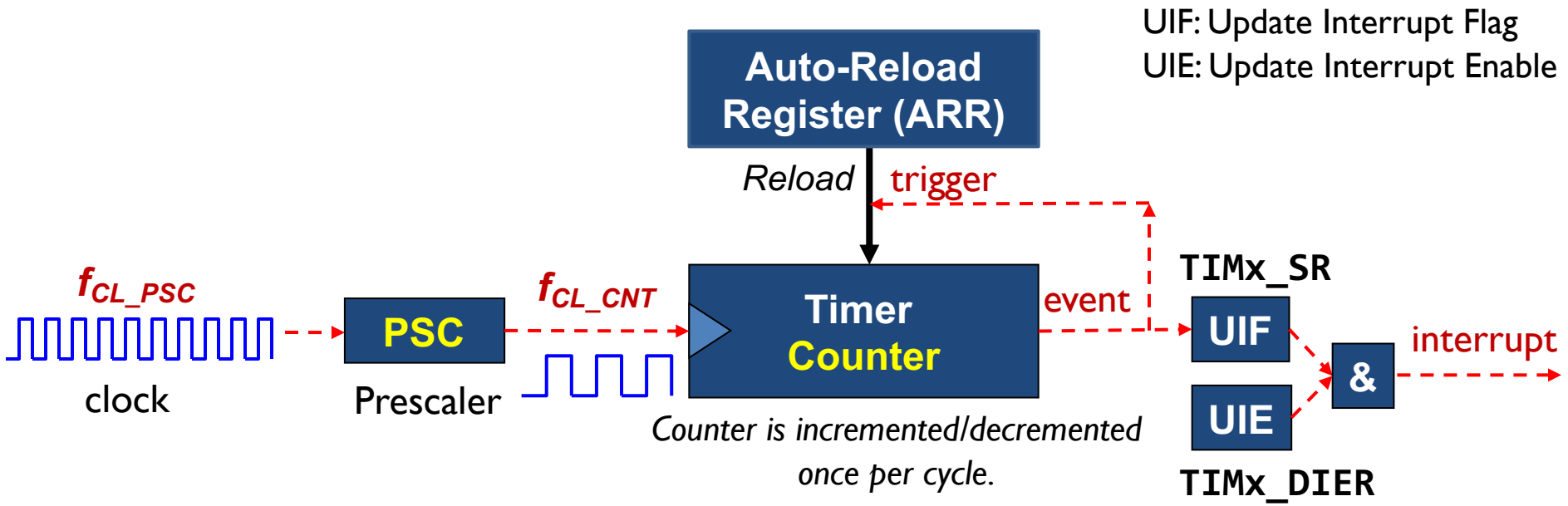
Timer

- Usually, embedded microprocessors have several elaborate timers that allow to
  - capture the current time or time differences, triggered by hardware or software events,
  - generate interrupts when a certain time is reached (stop watch, timeout),
  - generate interrupts when counters overflow,
  - generate periodic interrupts, for example in order to periodically execute tasks,
  - generate specific output signals, for example PWM (pulse width modulation).

Timer

- Free-run counter (independent of processor)
- Functions
  - **Input capture** (*to measure the pulse lengths of input signal*)
  - **Output compare** (*to control an output waveform, or to indicate when a period of time has elapsed*)
  - **Pulse-width modulation (PWM)** generation
  - **One-pulse mode** output (*allows counter to be started in response to a stimulus and to generate a pulse with a programmable length after a programmable delay*)

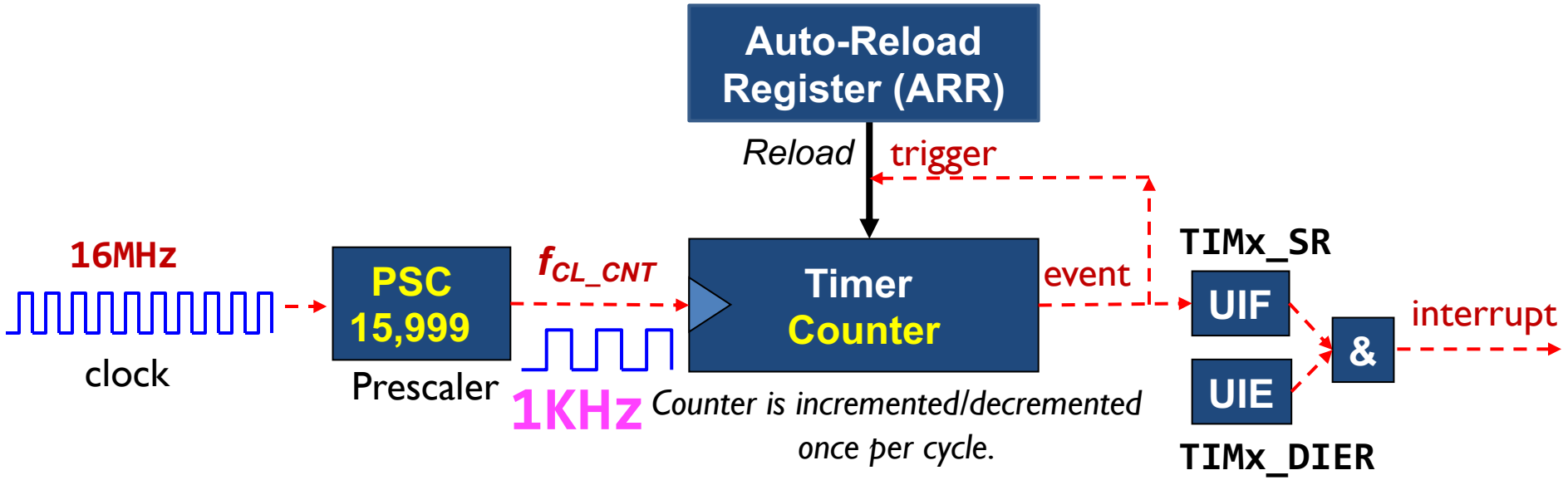
Timer: Clock



Frequency: $$f_{CK_CNT} = \frac{f_{CL_PSC}}{PSC + 1}$$

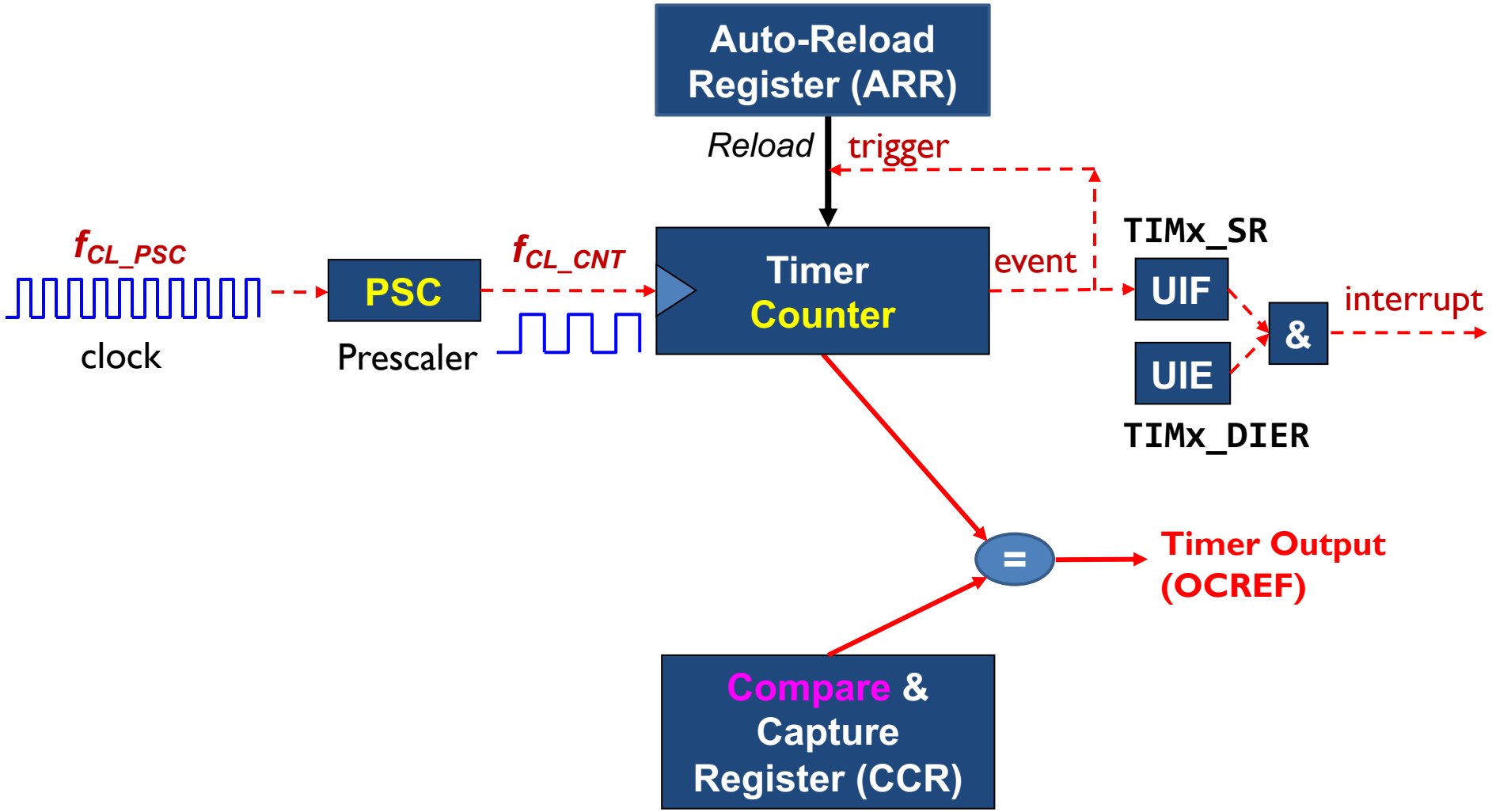
Software can change the prescaler (PSC) to allow the timer clocked at desired rate.

Timer: Clock

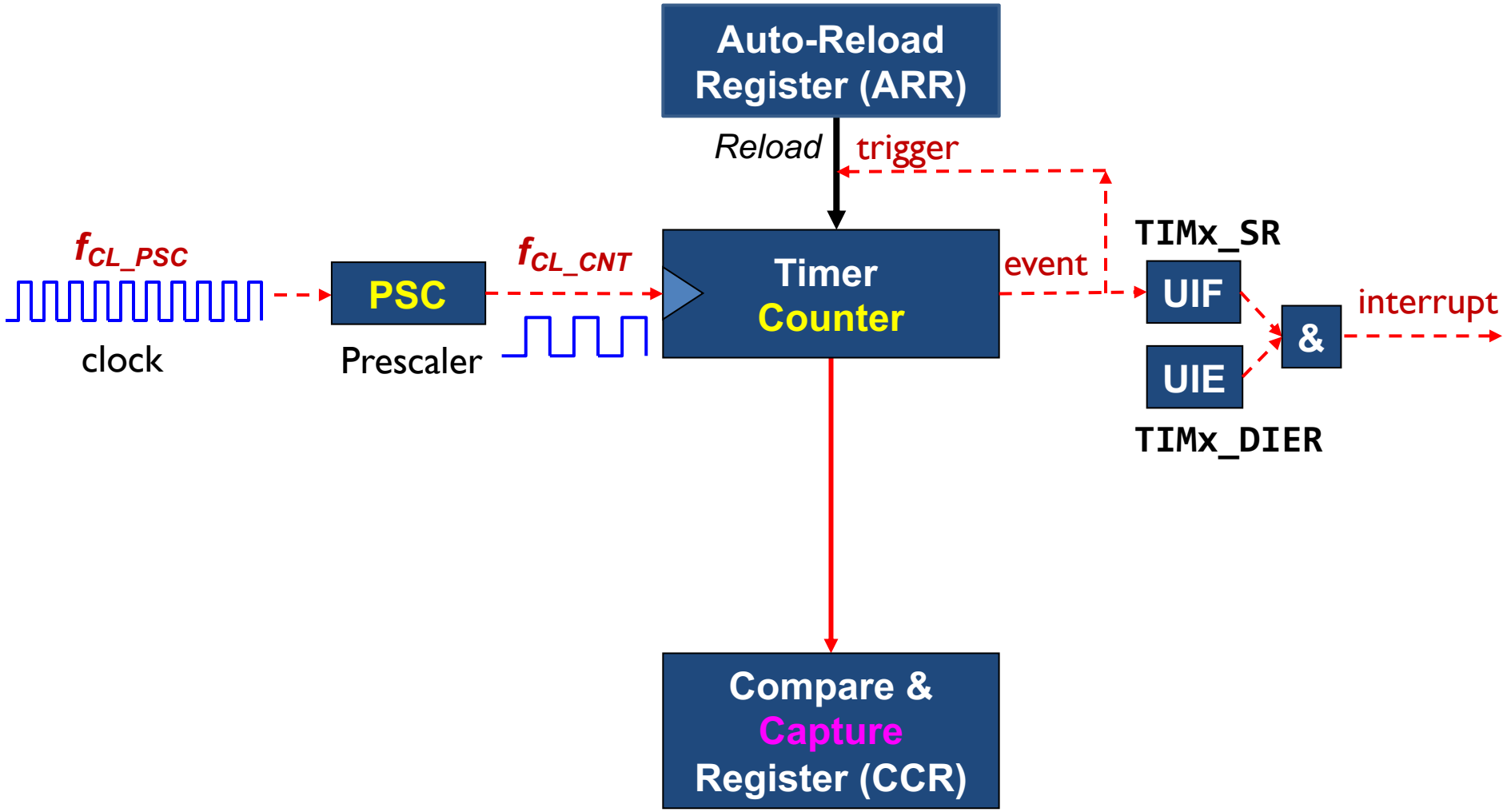


$$f_{CK_CNT} = \frac{16\ MHz}{15999 + 1} = 1\ KHz$$

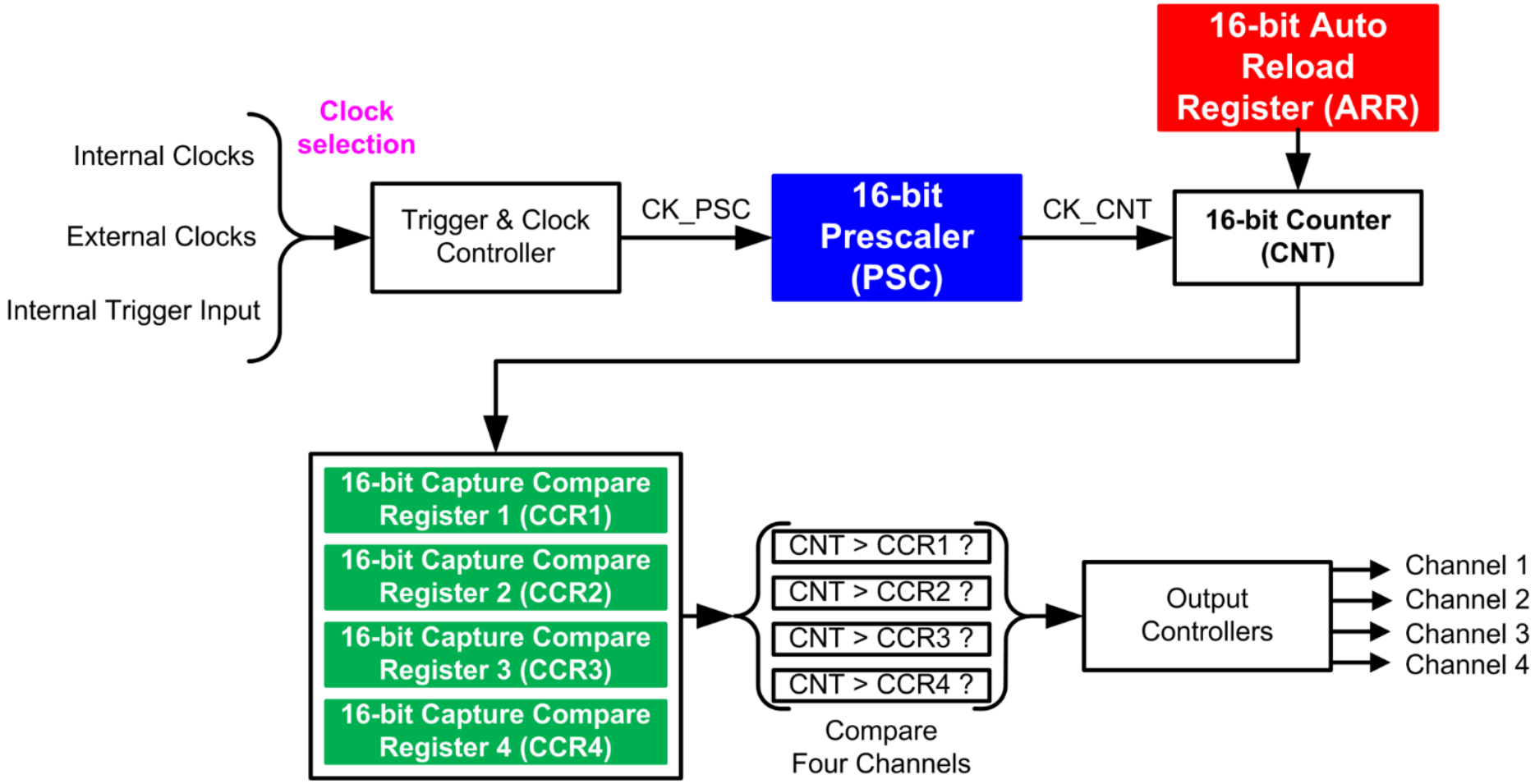
Timer: Output



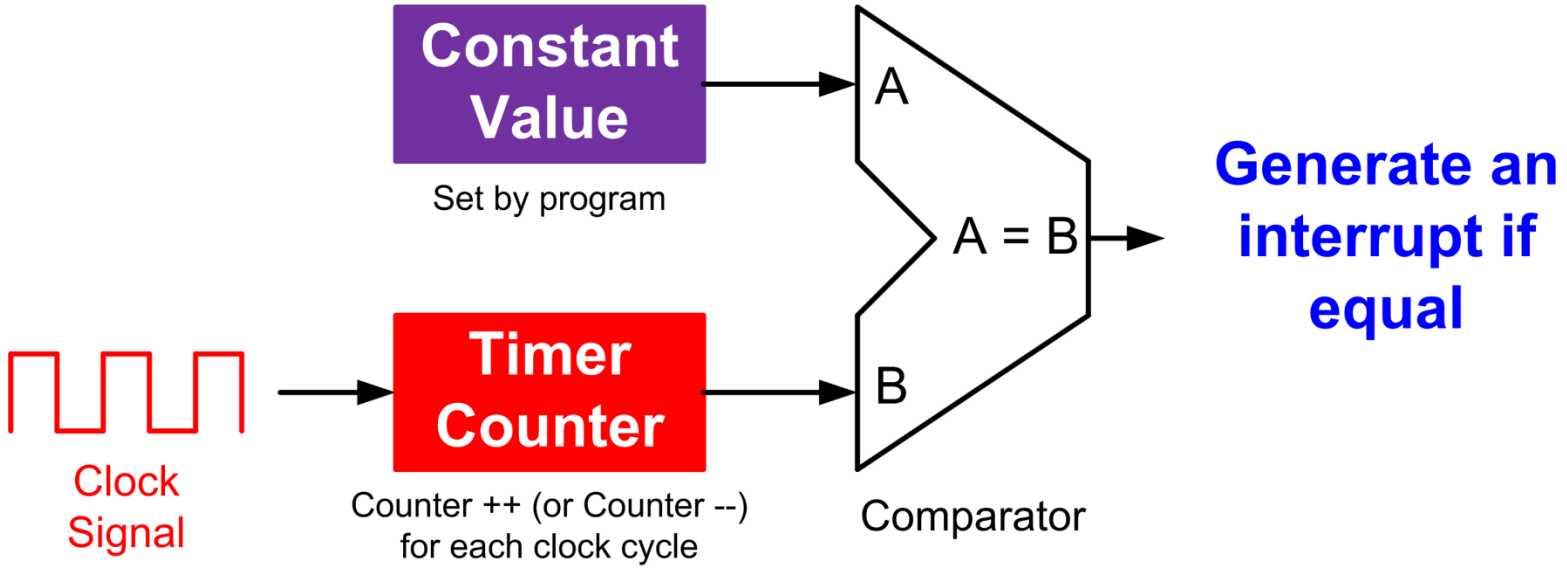
Timer: Input Capture



Multi-Channel Outputs



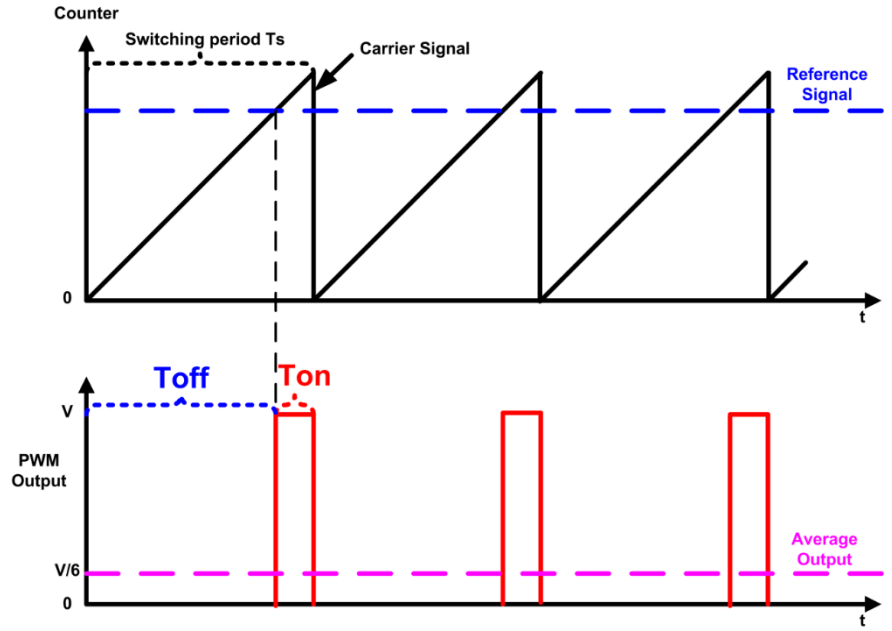
Output Compare



Output Compare Mode (OCM)	Timer Output (OCREF)
000	Frozen
001	High if CNT == CCR
010	Low if CNT == CCR
011	Toggle if CNT == CCR
100	Forced low (always low)
101	Forced high (always high)

PWM Mode

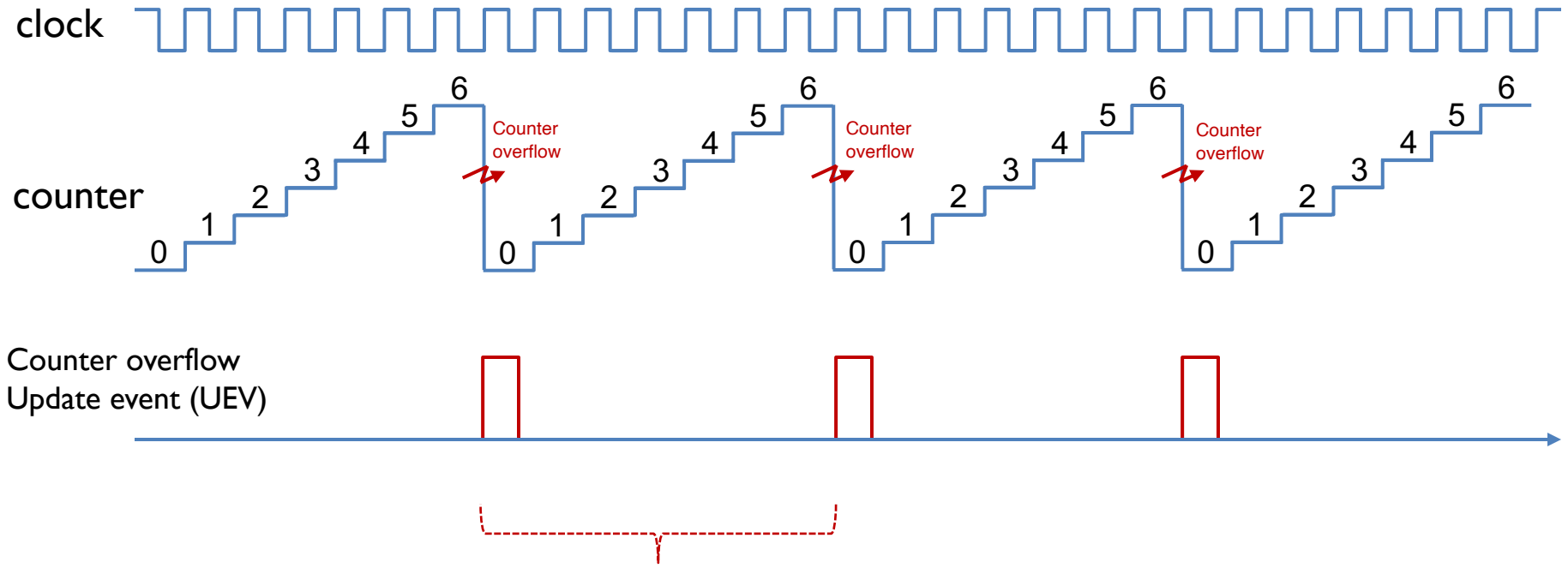
- **Pulse width modulation (PWM)** is a simple digital technique to control the value of an analog variable.
- It uses a rectangular waveform to quickly switch a voltage source on and off to produce a desired average voltage output.
- PWM has been widely used in a variety of applications, such as motor speed and torque control, digital encoding in telecommunications, and audio amplification.



Mode	Counter < Reference	Counter ≥ Reference
PWM mode 1 (Low True)	Active	Inactive
PWM mode 2 (High True)	**Inactive**	**Active**

Edge-aligned Mode (Up-counting)

ARR = 6, RCR = 0

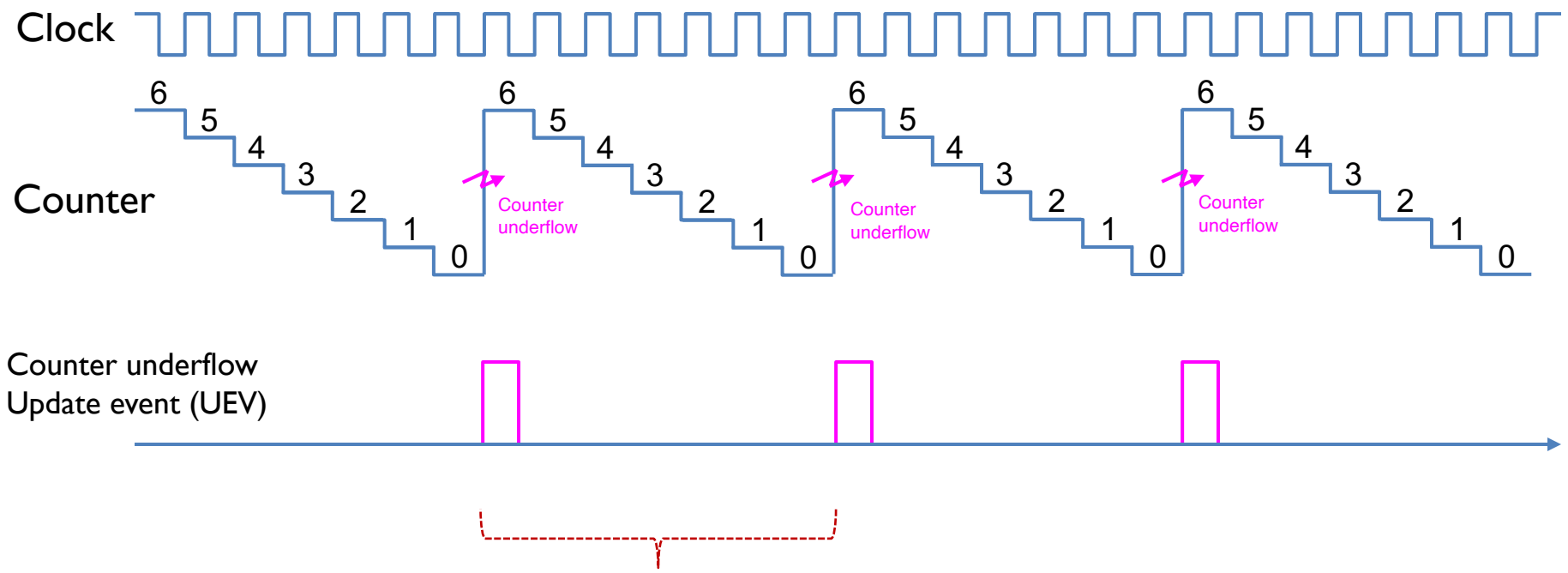


Period = (1 + ARR) * Clock Period
= 7 * Clock Period

Clock Period = $1/f_{CNT}$ $f_{CNT} = f_{SOURCE}/(1 + PSC)$

Edge-aligned Mode (down-counting)

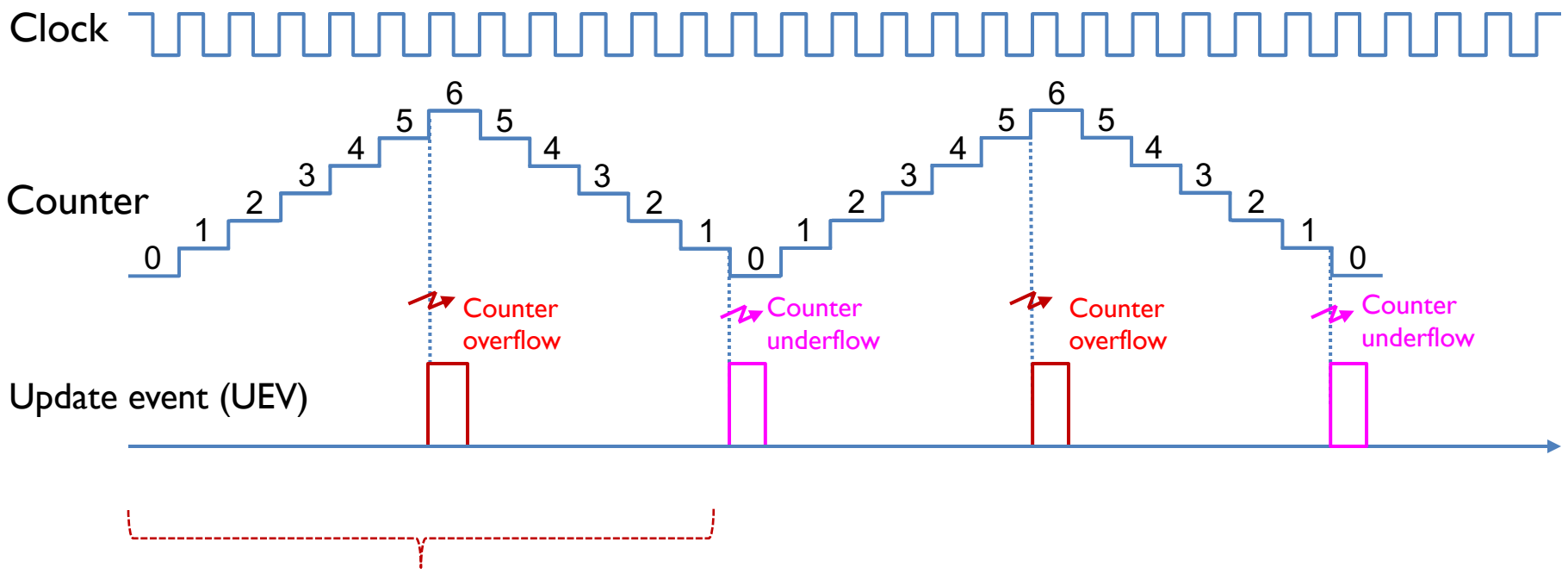
ARR = 6, RCR = 0




```
Period = (1 + ARR) * Clock Period
       = 7 * Clock Period
```

Center-aligned Mode

ARR = 6, RCR = 0

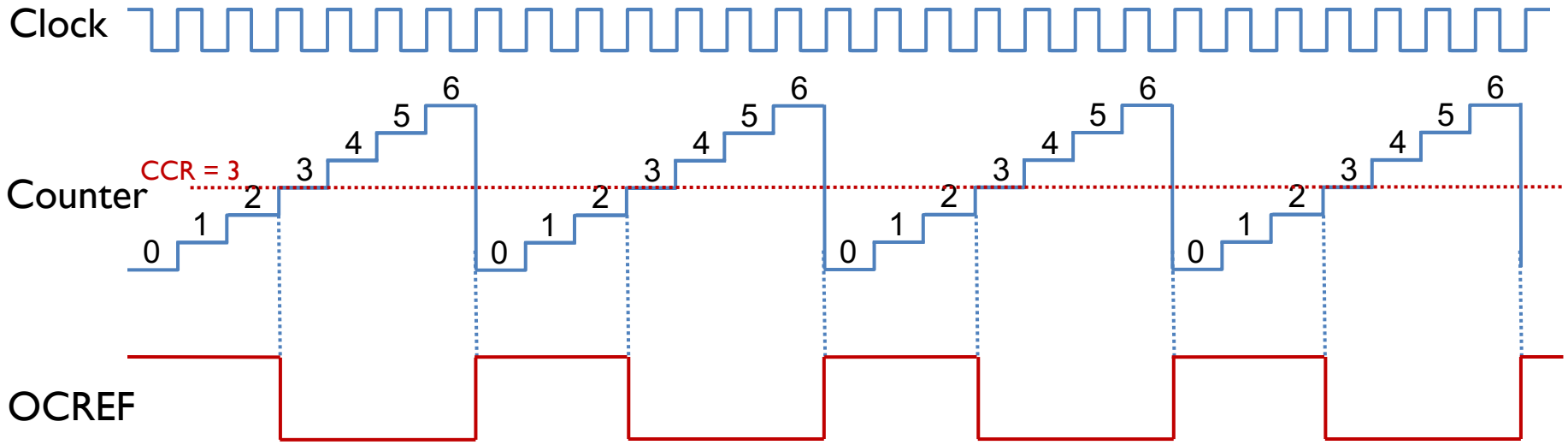



```
Period = 2 * ARR * Clock Period
       = 12 * Clock Period
```

PWM Mode 1 (Low-True)

Mode 1
Timer Output (OCREF) = { High if counter < CCR ; Low if counter ≥ CCR }

Upcounting mode, ARR = 6, CCR = 3, RCR = 0



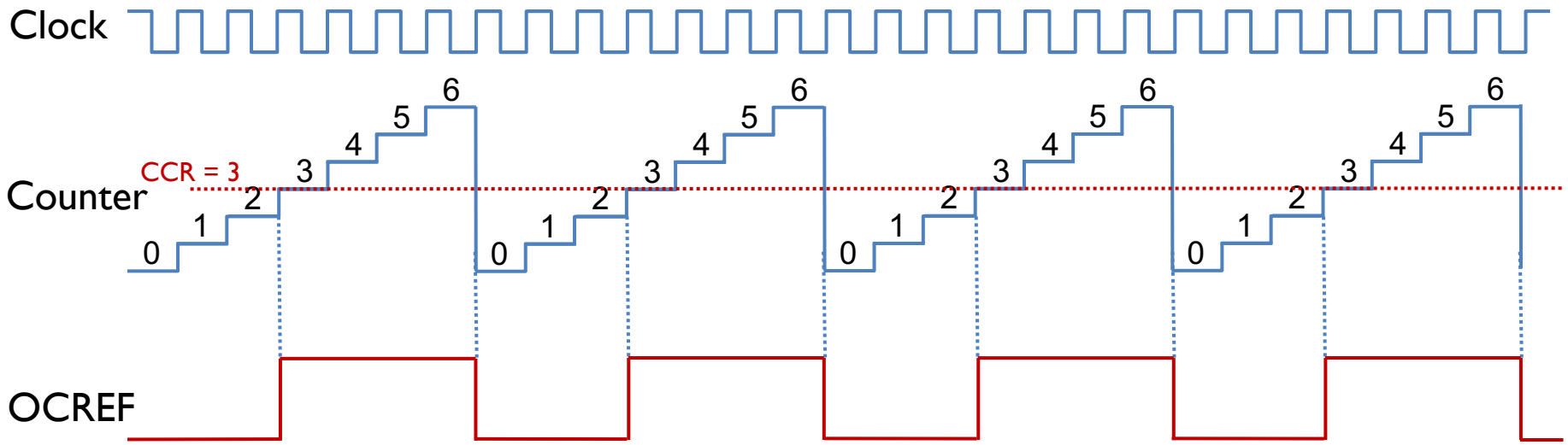
$$\text{Duty Cycle} = \frac{\text{CCR}}{\text{ARR} + 1} = \frac{3}{7}$$

$$\text{Period} = (1 + \text{ARR}) * \text{Clock Period} = 7 * \text{Clock Period}$$

PWM Mode 2 (High-True)

Mode 2
Timer Output (OCREF) = { Low if counter < CCR ; High if counter ≥ CCR }

Upcounting mode, ARR = 6, CCR = 3, RCR = 0



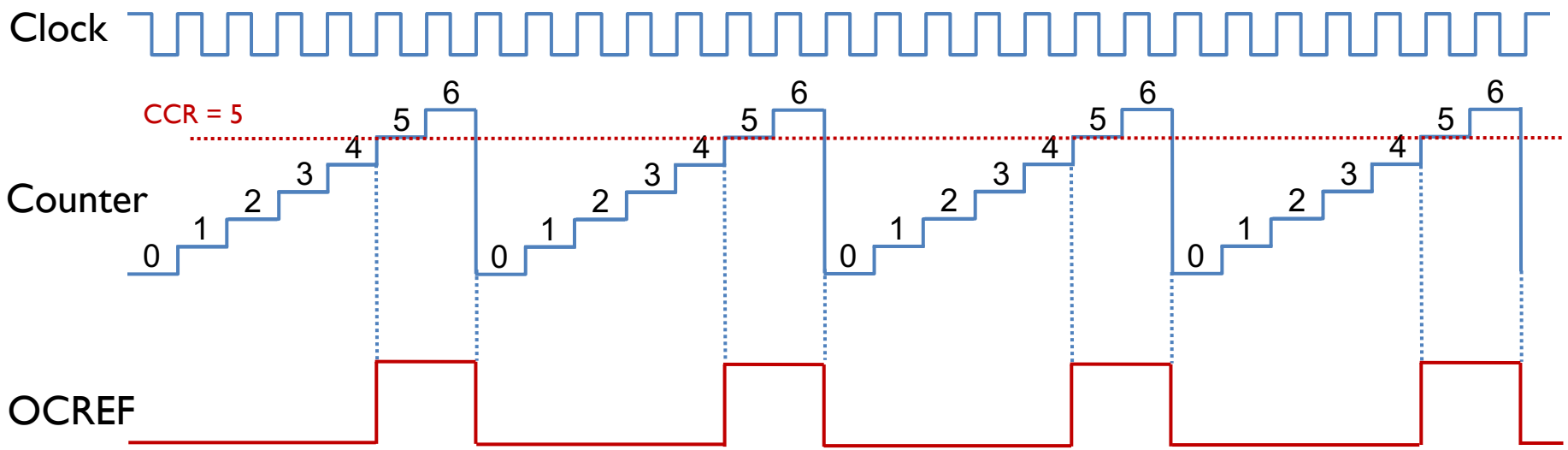
$$\text{Duty Cycle} = 1 - \frac{CCR}{ARR + 1} = \frac{4}{7}$$

$$\text{Period} = (1 + ARR) * \text{Clock Period} = 7 * \text{Clock Period}$$

PWM Mode 2 (High-True)

Mode 2
Timer Output (OCREF) =
- Low if counter < CCR
- High if counter ≥ CCR

Upcounting mode, ARR = 6, CCR = 5, RCR = 0



$$\text{Duty Cycle} = 1 - \frac{CCR}{ARR + 1} = \frac{2}{7}$$

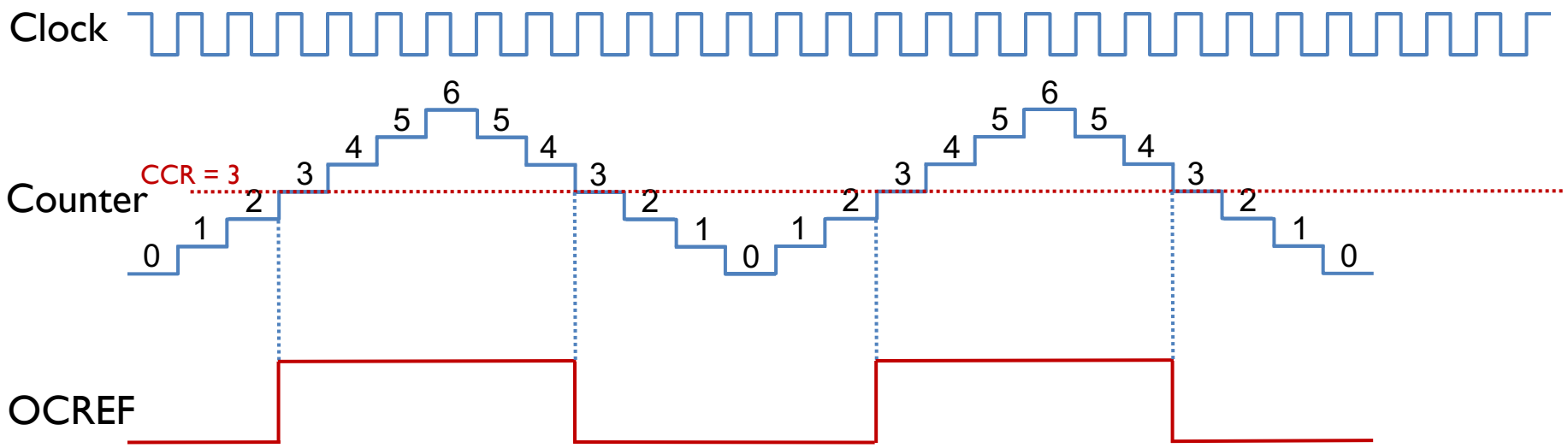
$$\text{Period} = (1 + ARR) * \text{Clock Period} = 7 * \text{Clock Period}$$

PWM Mode 2 (High-True)

Mode 2

Timer Output (OCREF) = { Low if counter < CCR ; High if counter ≥ CCR

Center-aligned mode, ARR = 6, CCR = 3, RCR = 0



$$\text{Duty Cycle} = 1 - \frac{CCR}{ARR} = \frac{1}{2}$$

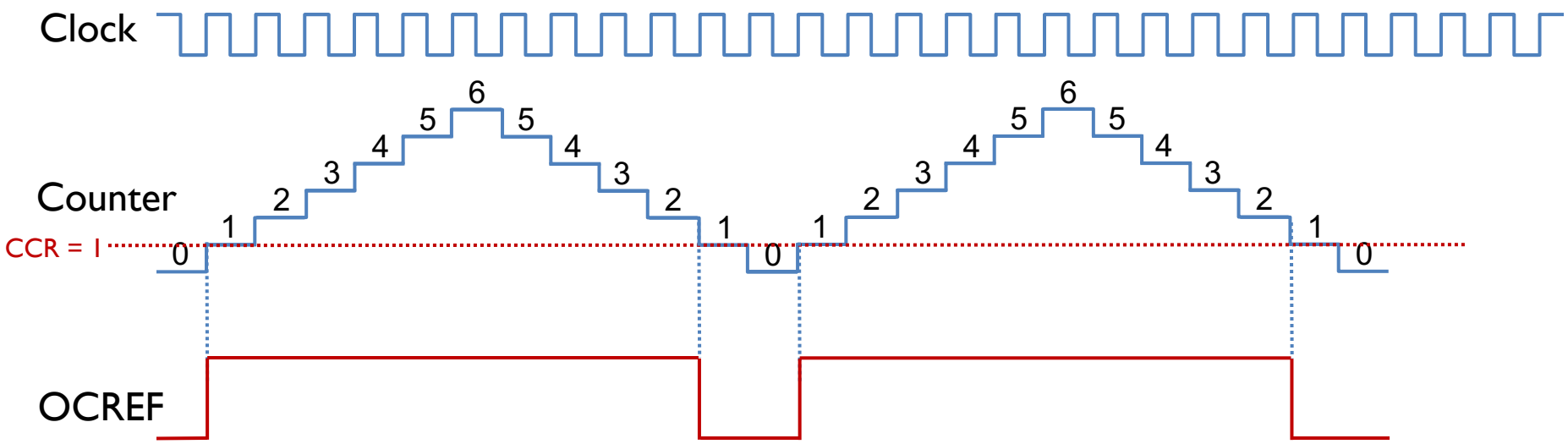
$$\text{Period} = 2 * ARR * \text{Clock Period} = 12 * \text{Clock Period}$$

PWM Mode 2 (High-True)

Mode 2

Timer Output (OCREF) =
- Low if counter < CCR
- High if counter ≥ CCR

Center-aligned mode, ARR = 6, CCR = 1, RCR = 0



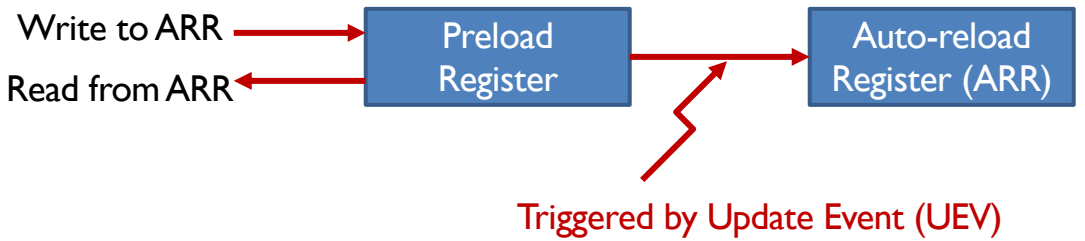
$$\text{Duty Cycle} = 1 - \frac{CCR}{ARR} = \frac{5}{6}$$

$$\text{Period} = 2 * ARR * \text{Clock Period} = 12 * \text{Clock Period}$$

Auto-Reload Register (ARR)

- Auto-Reload Preload Enable (ARPE) bit in TIMx_CR1

ARPE = 1 (Syn Update)



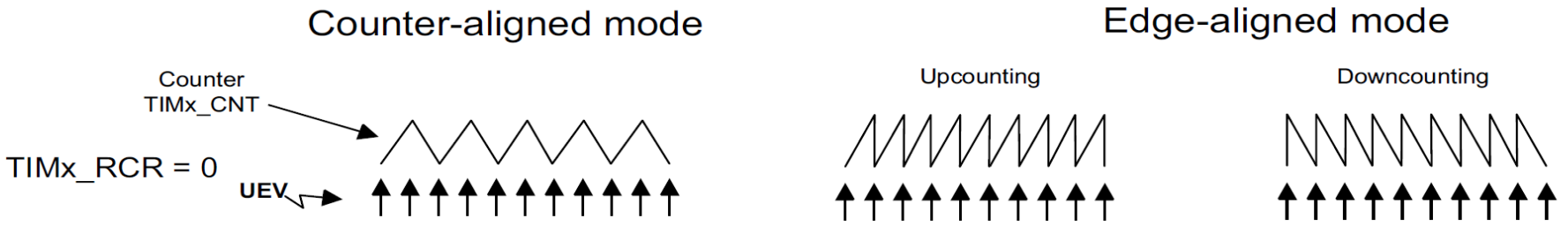
If UDIS bit in TIMx_CR1 is 1, UEV event is disabled.

ARPE = 0 (Asyn Update)

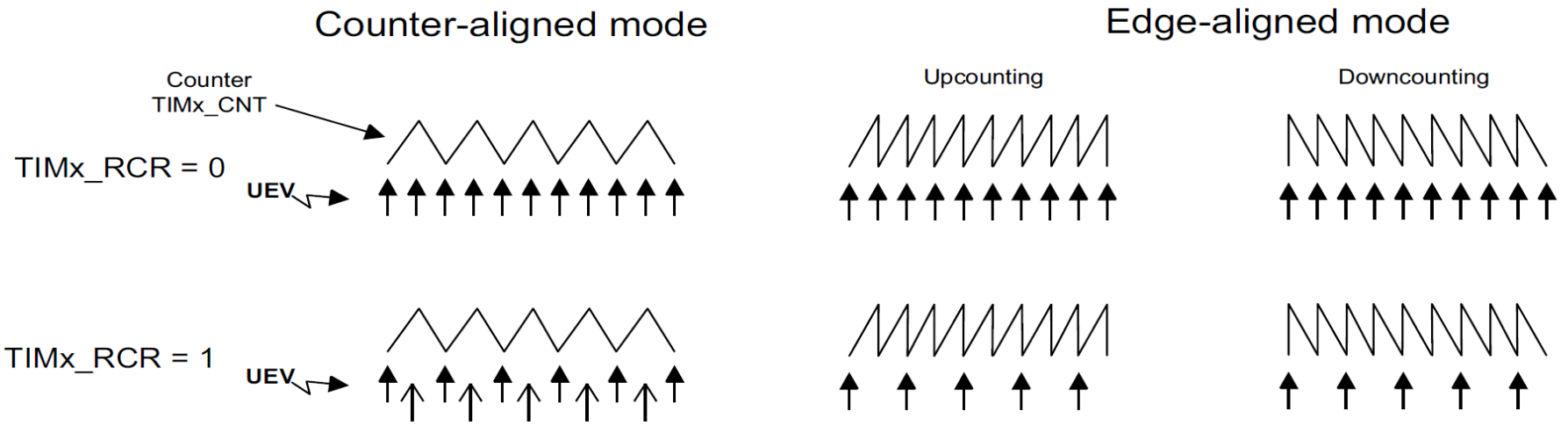


Repetition Counter Register (RCR)

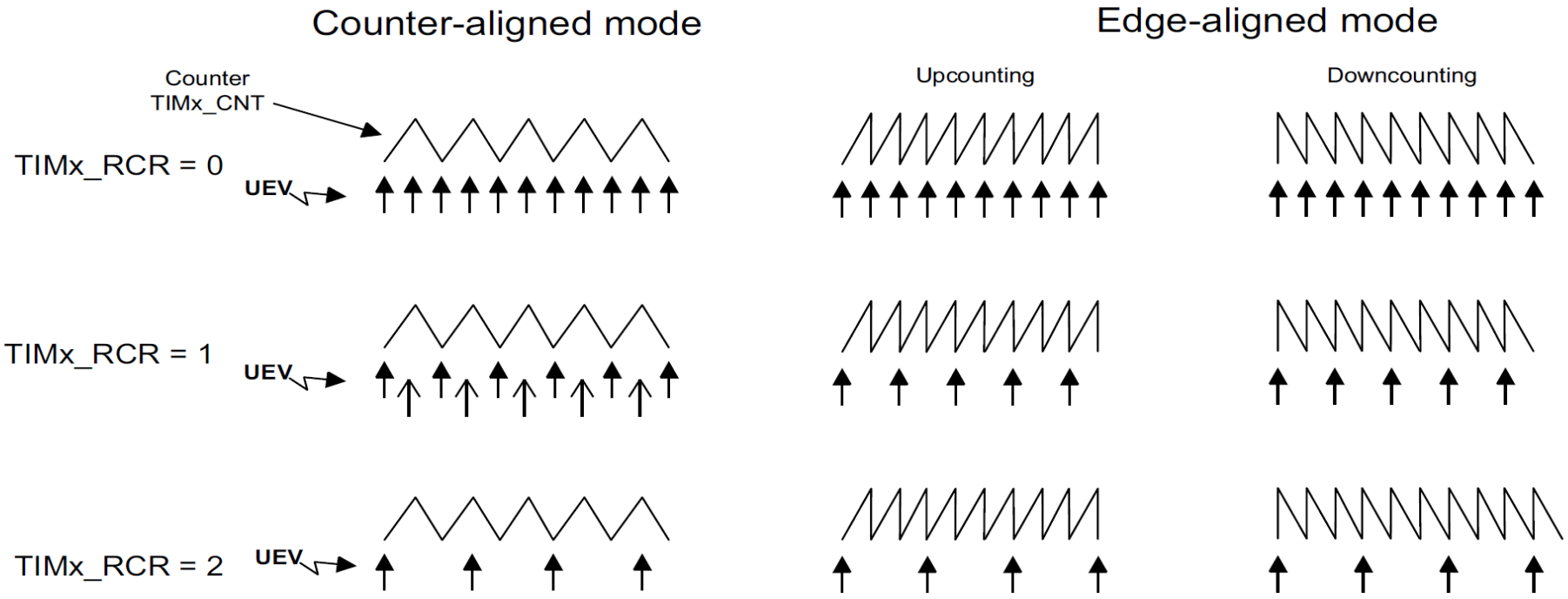
UDIS bit = 0



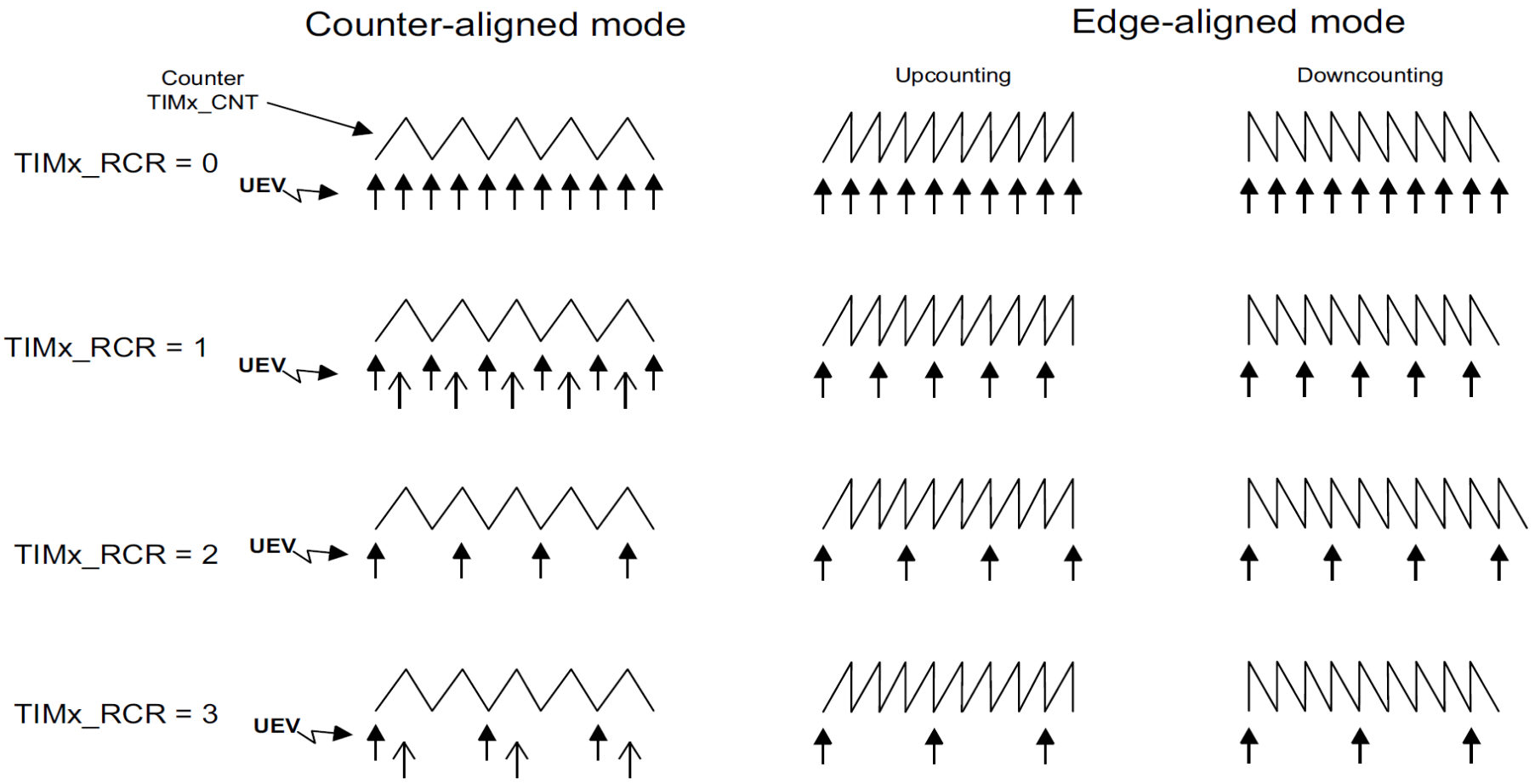
Repetition Counter Register (RCR)



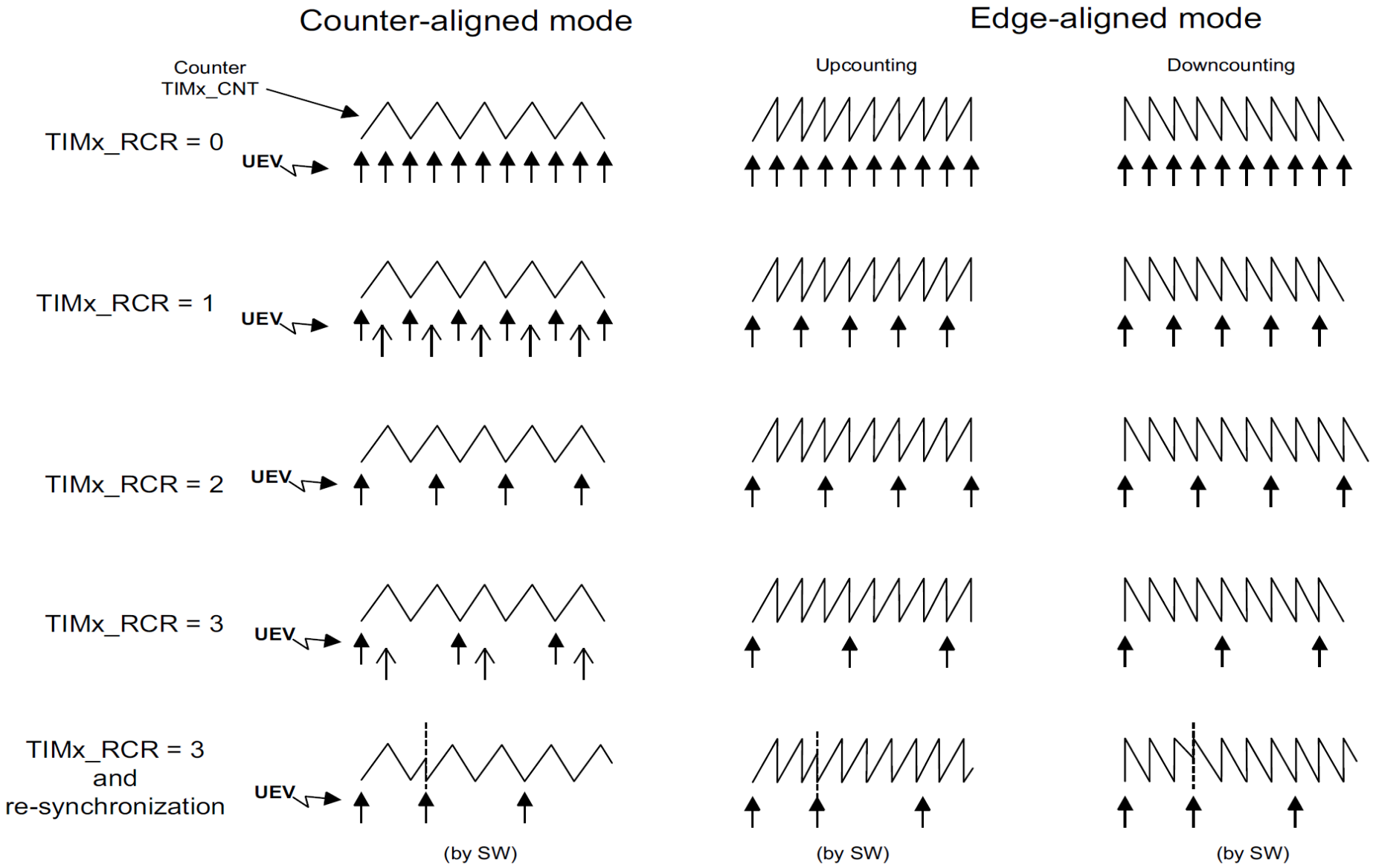
Repetition Counter Register (RCR)



Repetition Counter Register (RCR)



Repetition Counter Register (RCR)



PWM Output Polarity

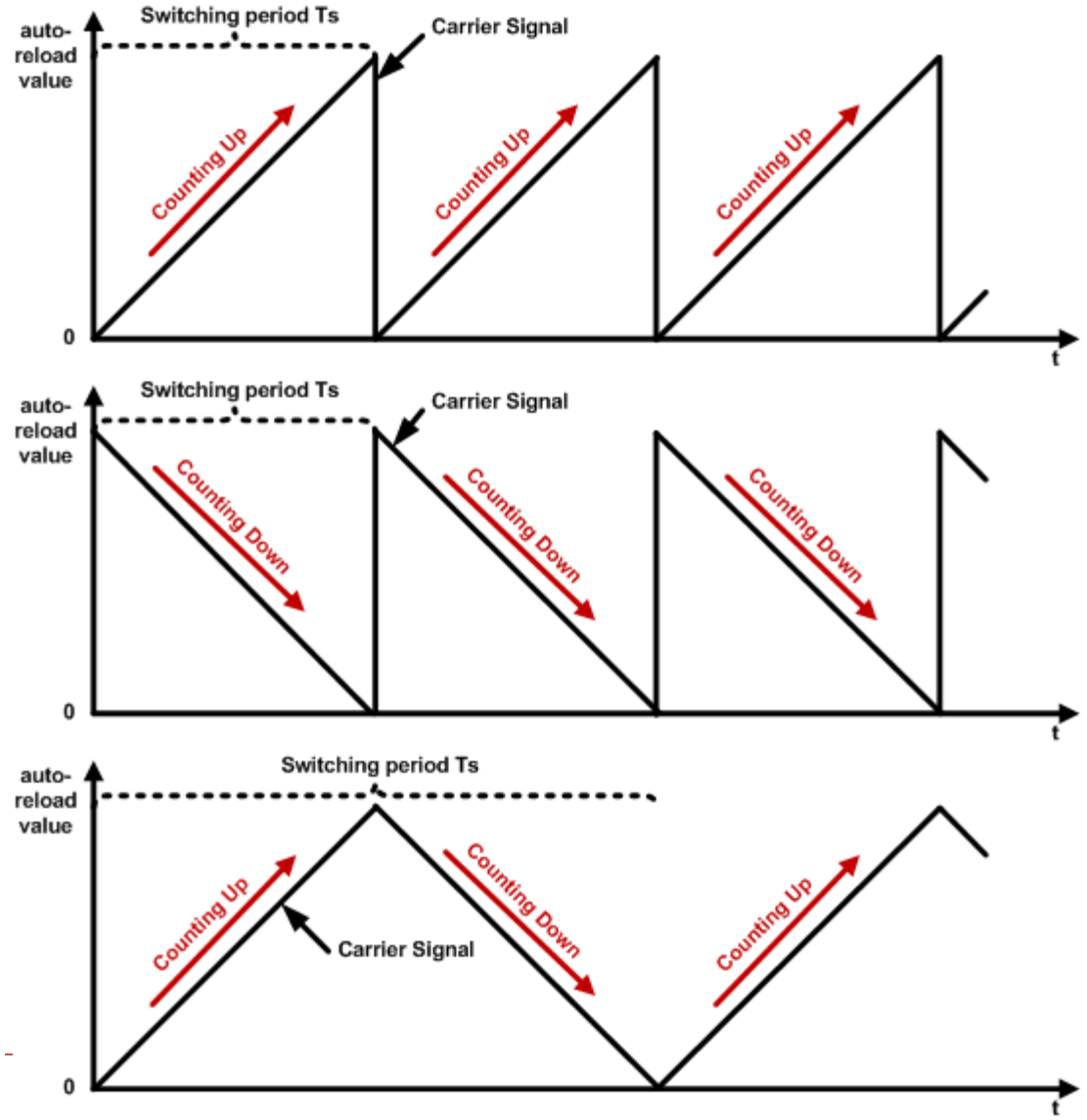
Mode	Counter < CCR	Counter ≥ CCR
PWM mode 1 (Low True)	Active	Inactive
PWM mode 2 (High True)	Inactive	Active

Output Polarity:

- Software can program the CCxP bit in the TIMx_CCER register

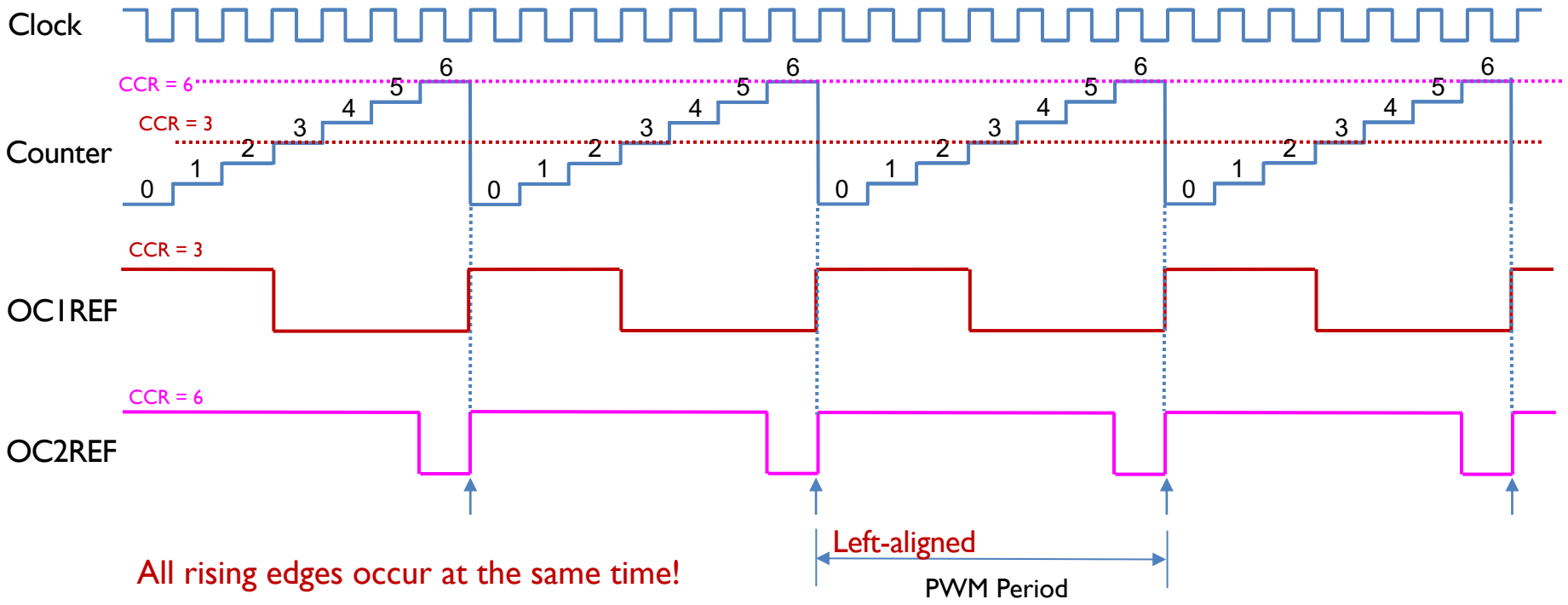
	Active	Inactive
Active High	High Voltage	Low Voltage
Active Low	Low Voltage	High Voltage

Counting up, down, center



Up-Counting: Left Edge-aligned

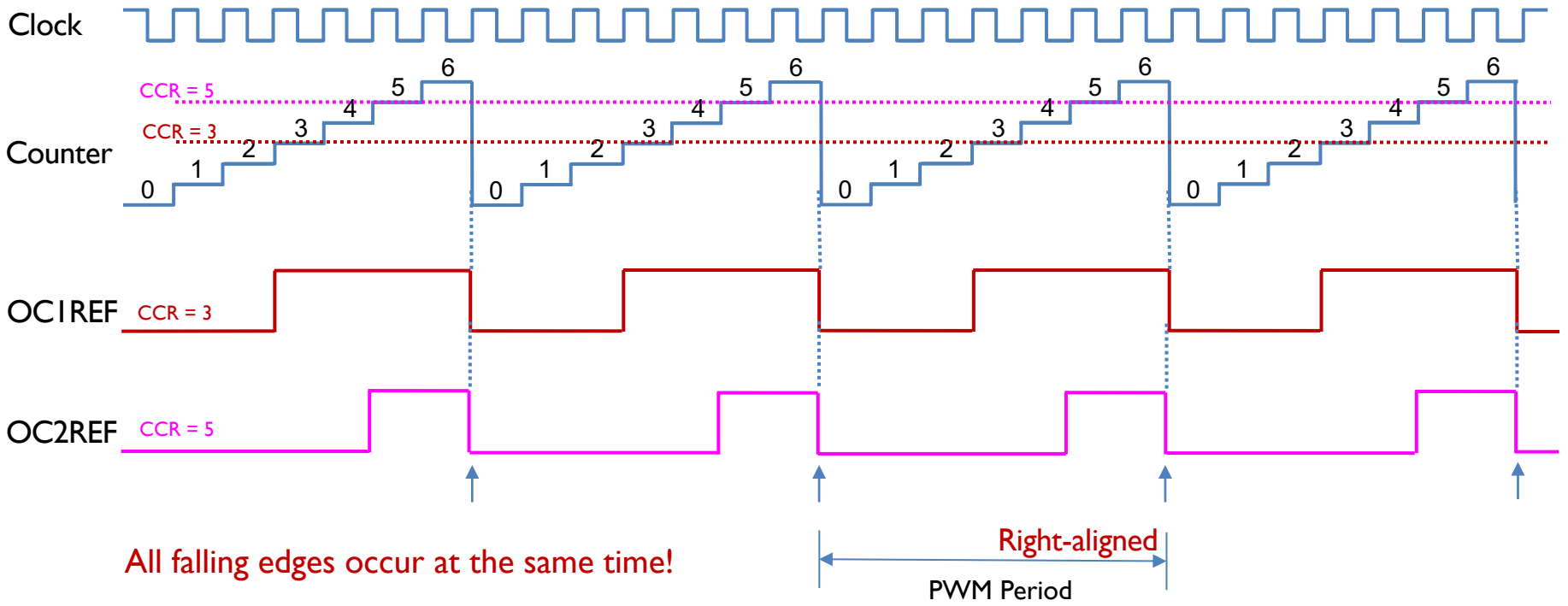
Upcounting mode, ARR = 6, CCR = 3, RCR = 0



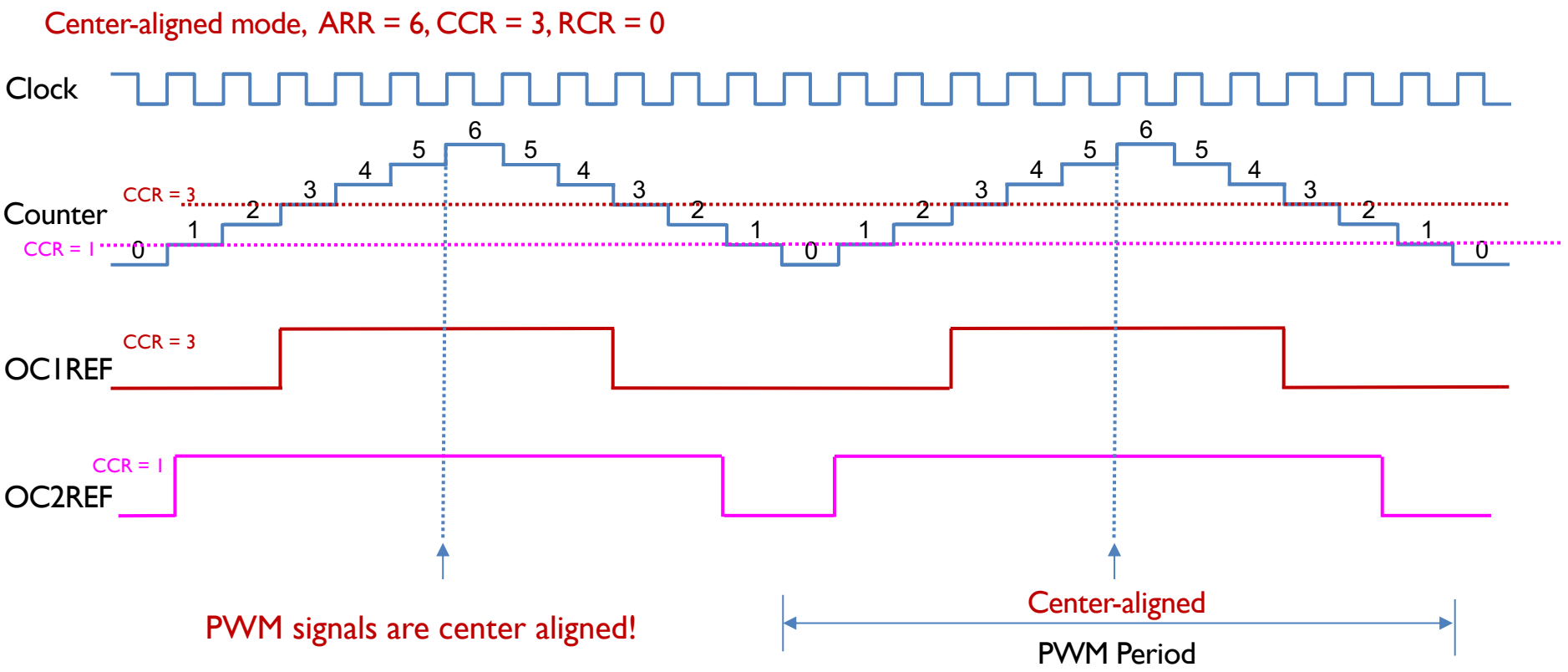
All rising edges occur at the same time!

PWM Mode 2: Right Edge-aligned

Upcounting mode, ARR = 6, CCR = 3, RCR = 0



PWM Mode 2: Center Aligned



Summary: Timer Output

Up-counting or Down-counting:

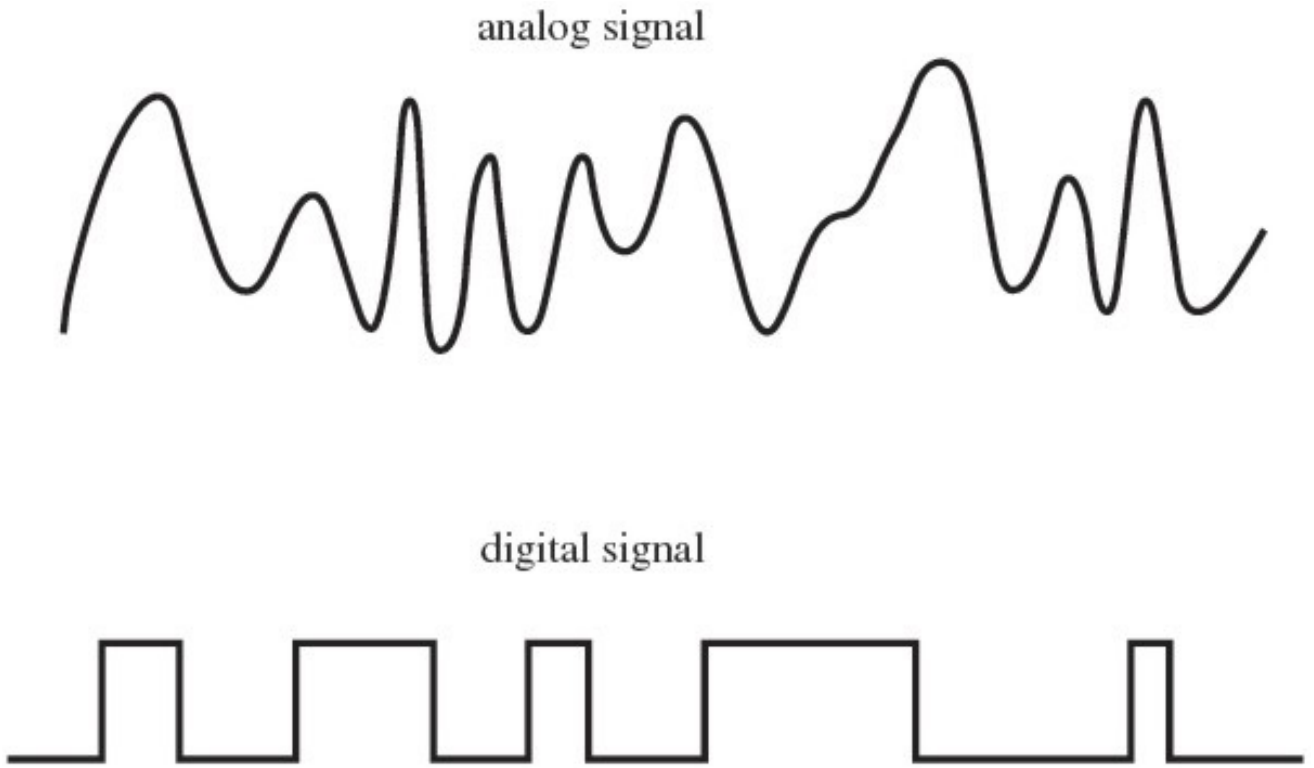
$$Output\ Signal\ Period = \frac{1}{f_{CNT}} \times (ARR + 1)$$

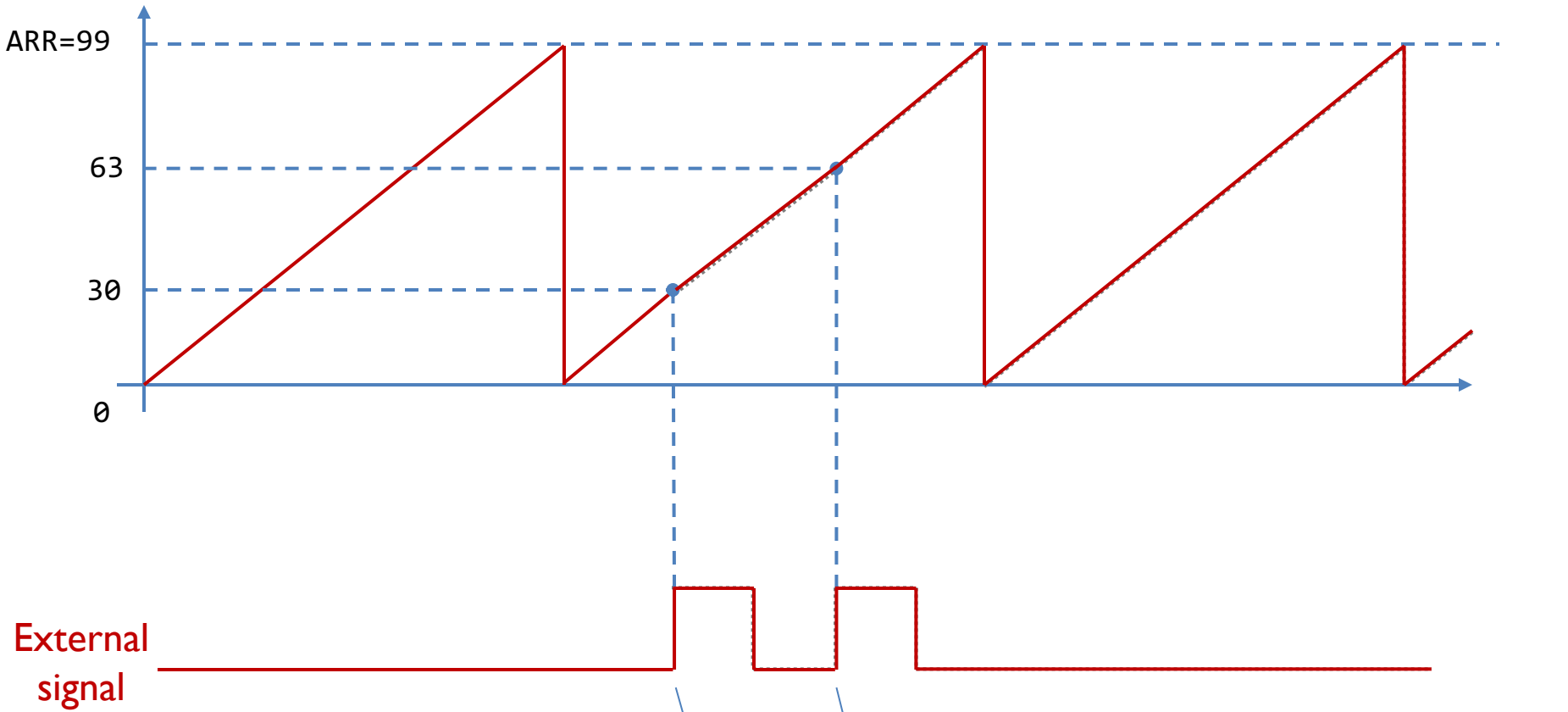
$$f_{CNT} = \frac{f_{SOURCE}}{PSC + 1}$$

$$Output\ Signal\ Period = \frac{(ARR + 1)(PSC + 1)}{f_{SOURCE}}$$

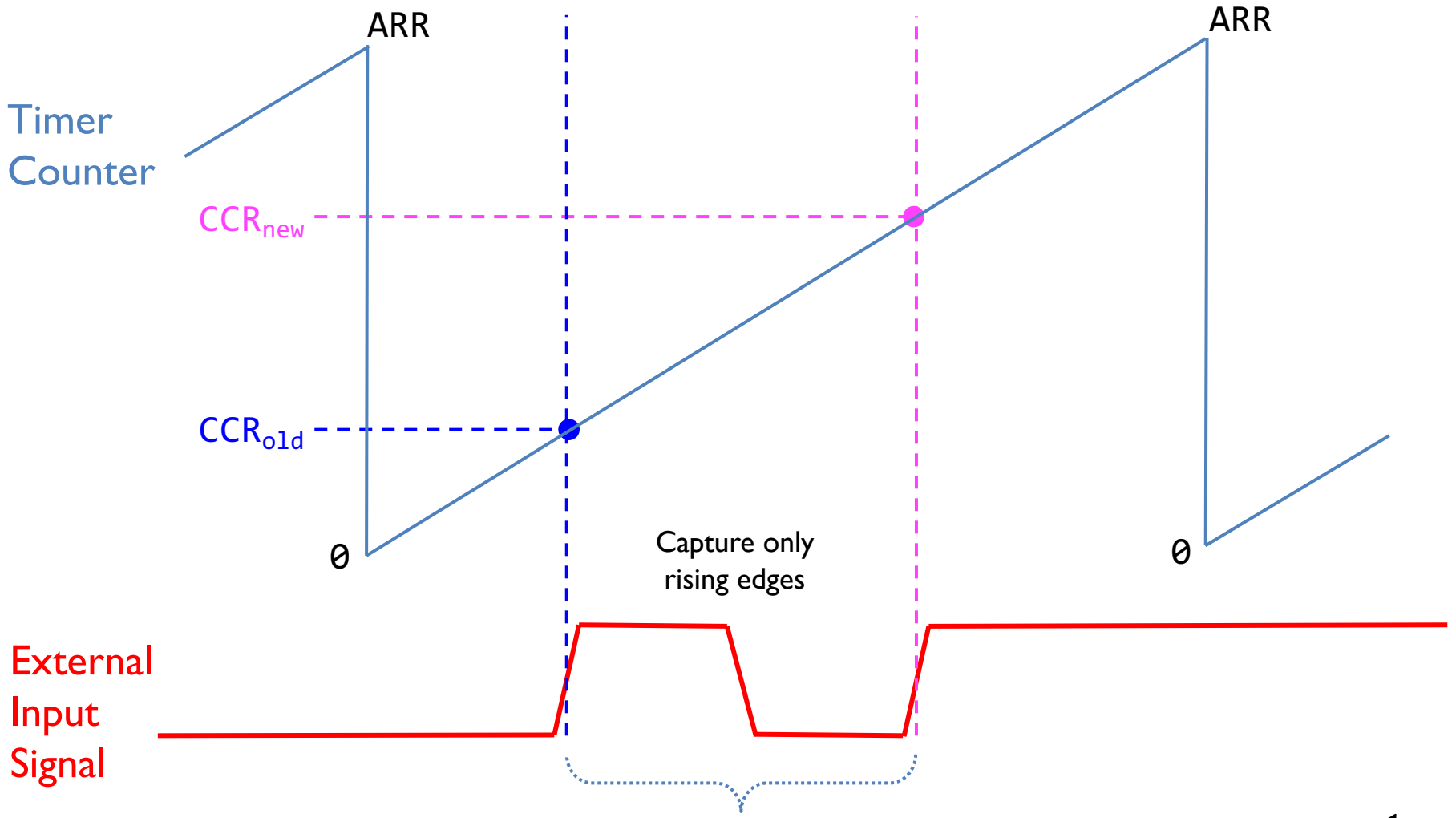
Input Capture

- Capture the timestamp of the occurrence of a rising or a falling edge transition in a digital signal





Time span between two events = 63 – 30 = 33 time units

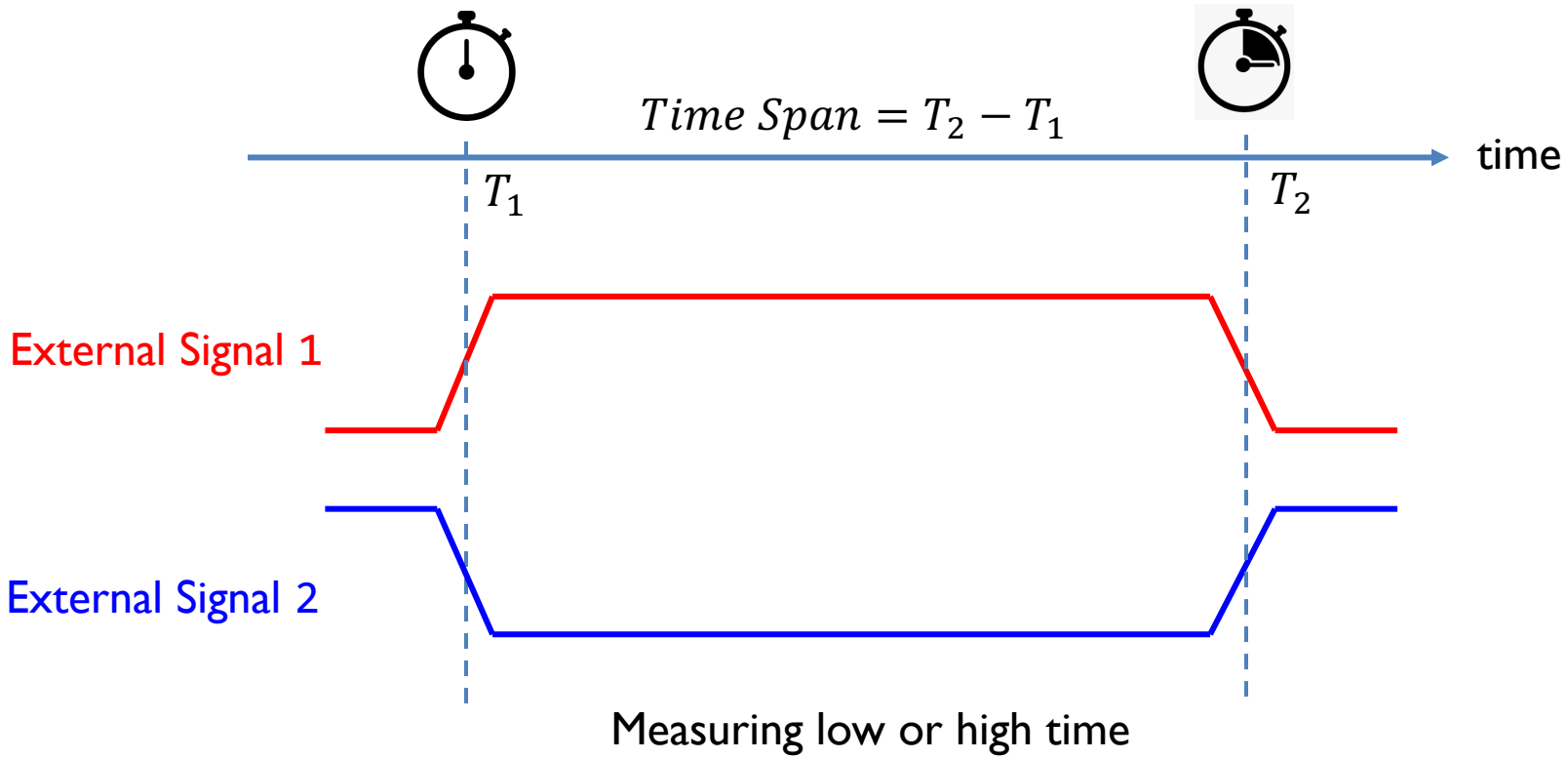


$$Time\ Span = (CCR_{new} - CCR_{old}) \times \frac{1}{f_{CL_CNT}}$$

$$f_{CK_CNT} = \frac{f_{CL_PSC}}{PSC + 1}$$

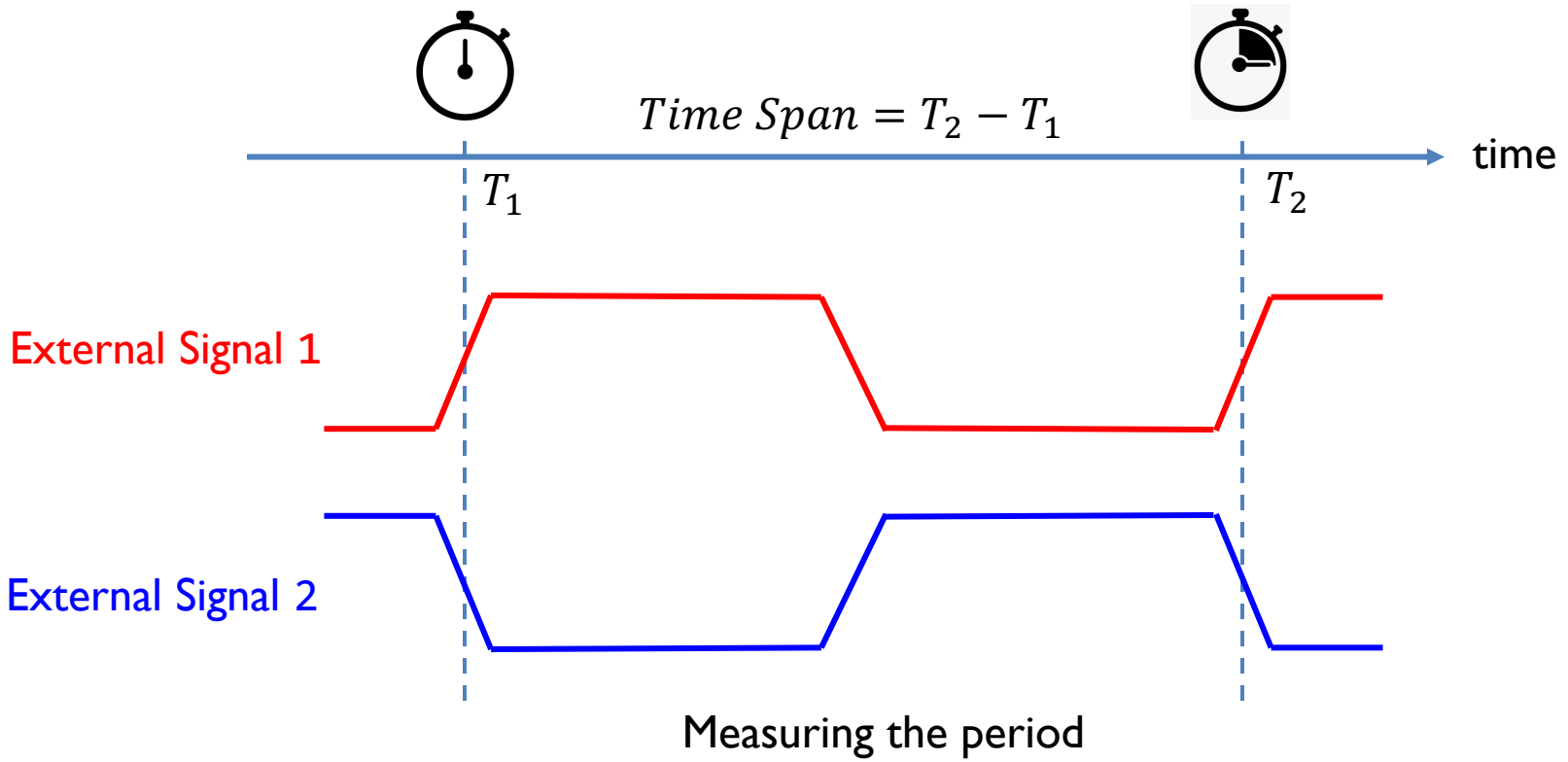
Input Capture

- Record the timestamp of an external event
- Capture both rising and falling edges



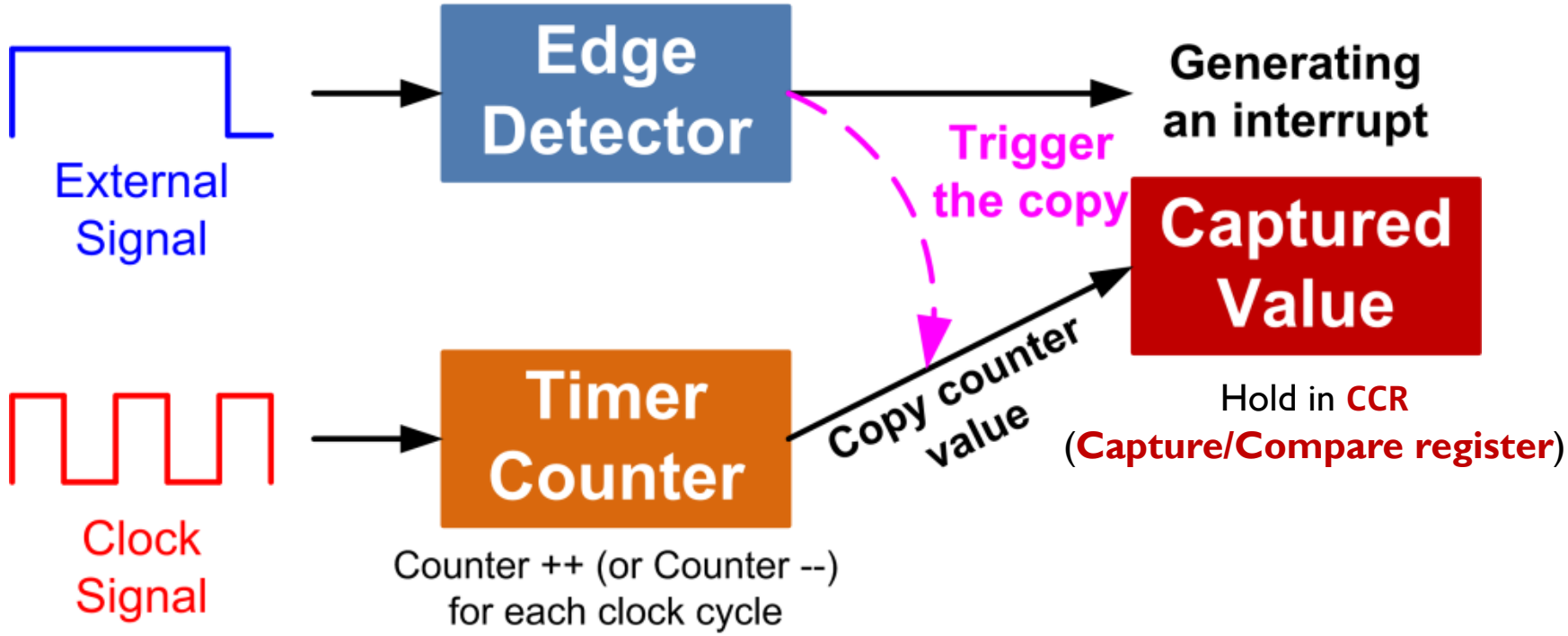
Input Capture

- Capture only rising edges or only falling edges



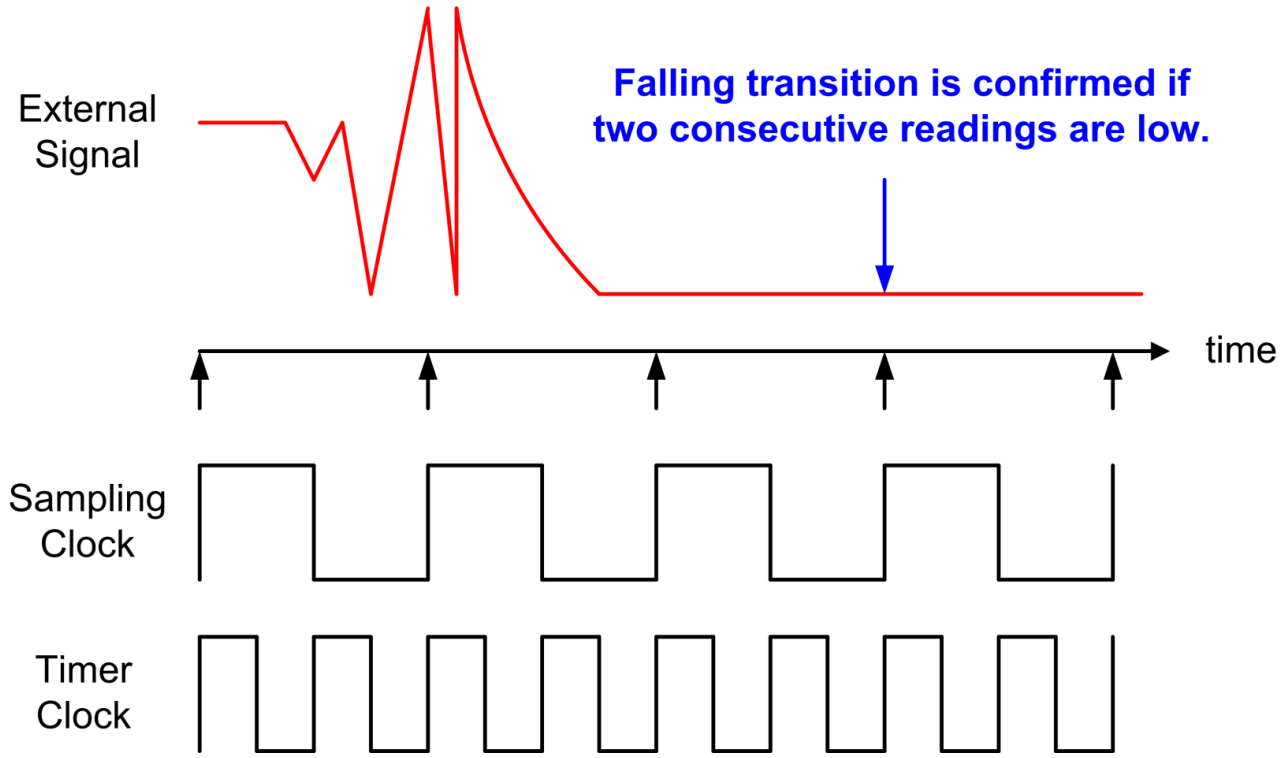
Input Capture

- Latch the counter value into CCR after a transition is detected
- If enabled, generate an interrupt to inform the processor to read CCR



Input Filtering

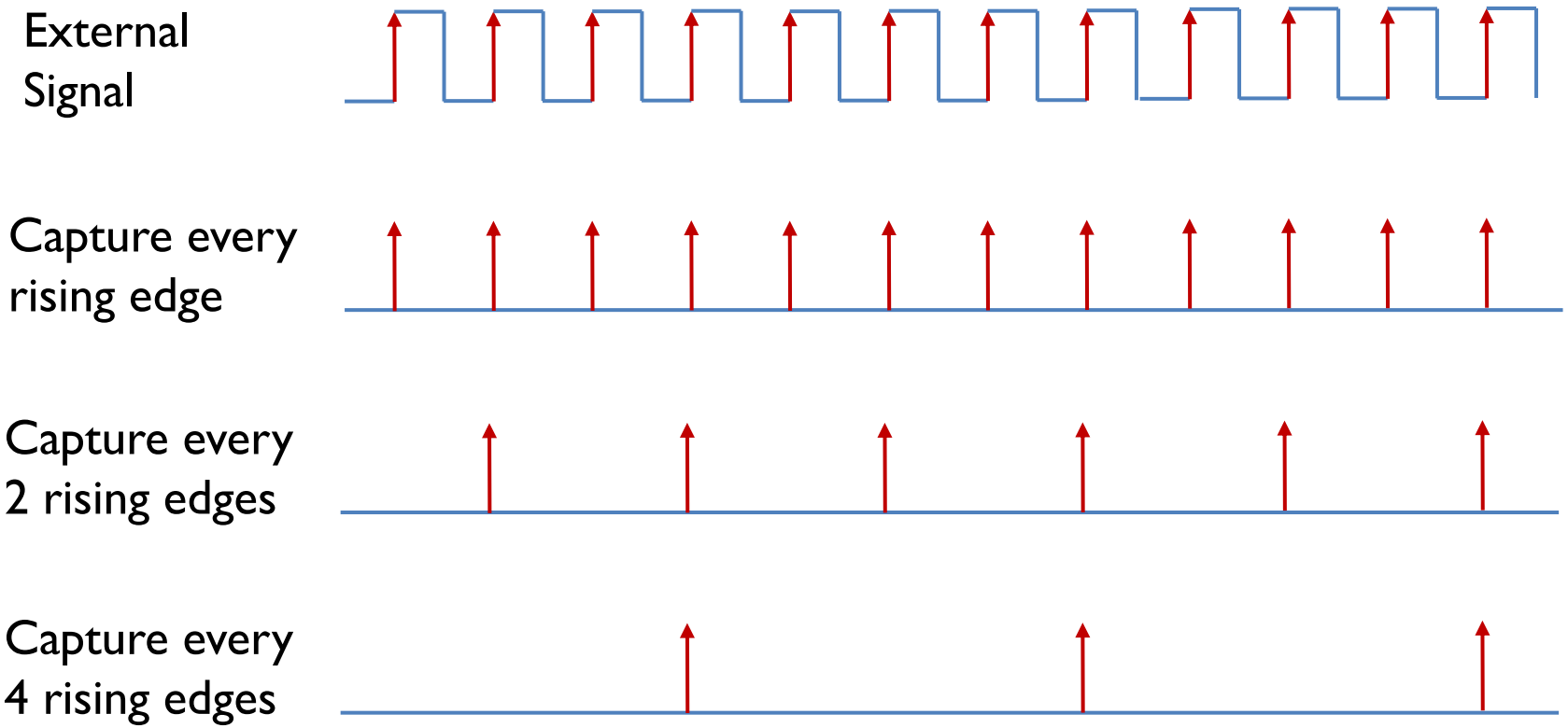
- Digital filter for debouncing and noise removal
  - N consecutive samples with the new level



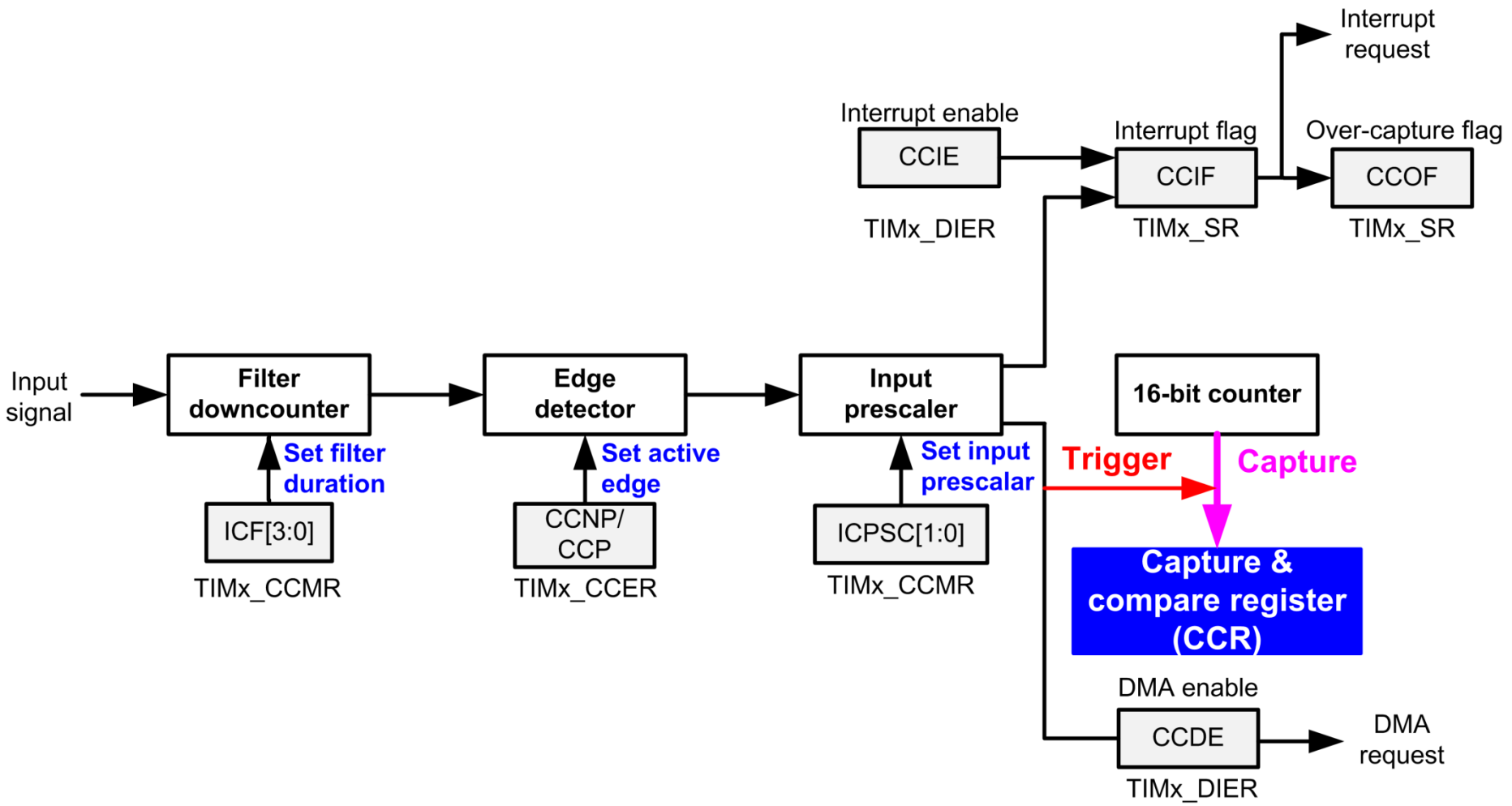
Filter acceptance = 2

Input Prescaler

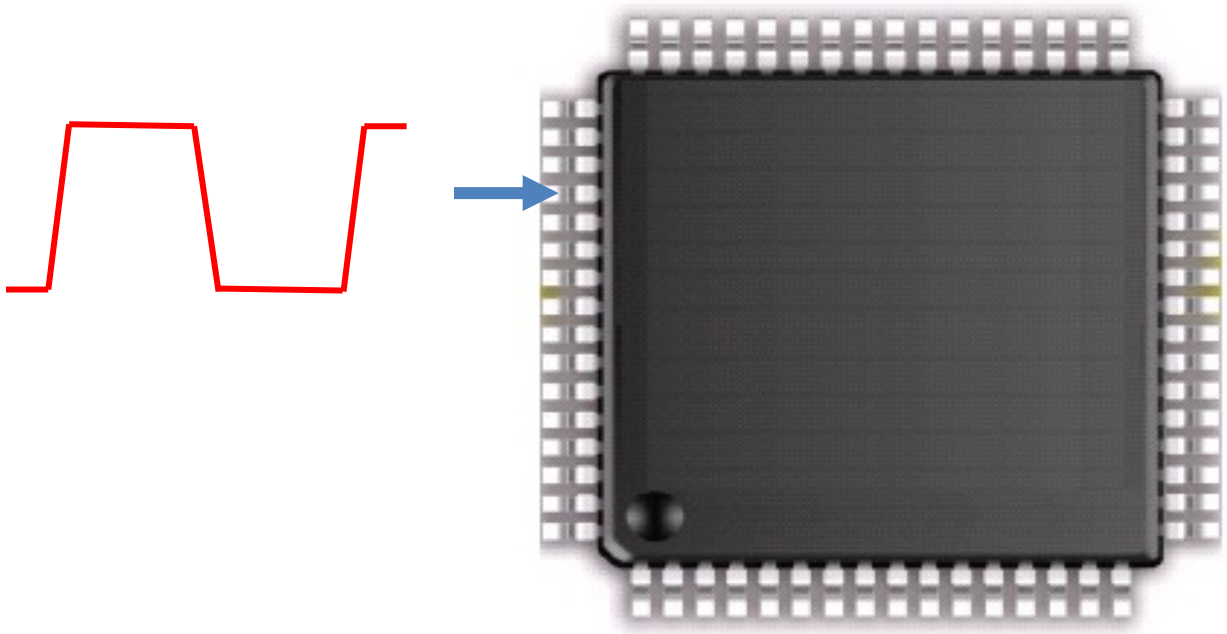
- Assume capture only rising edge



Input Capture Diagram



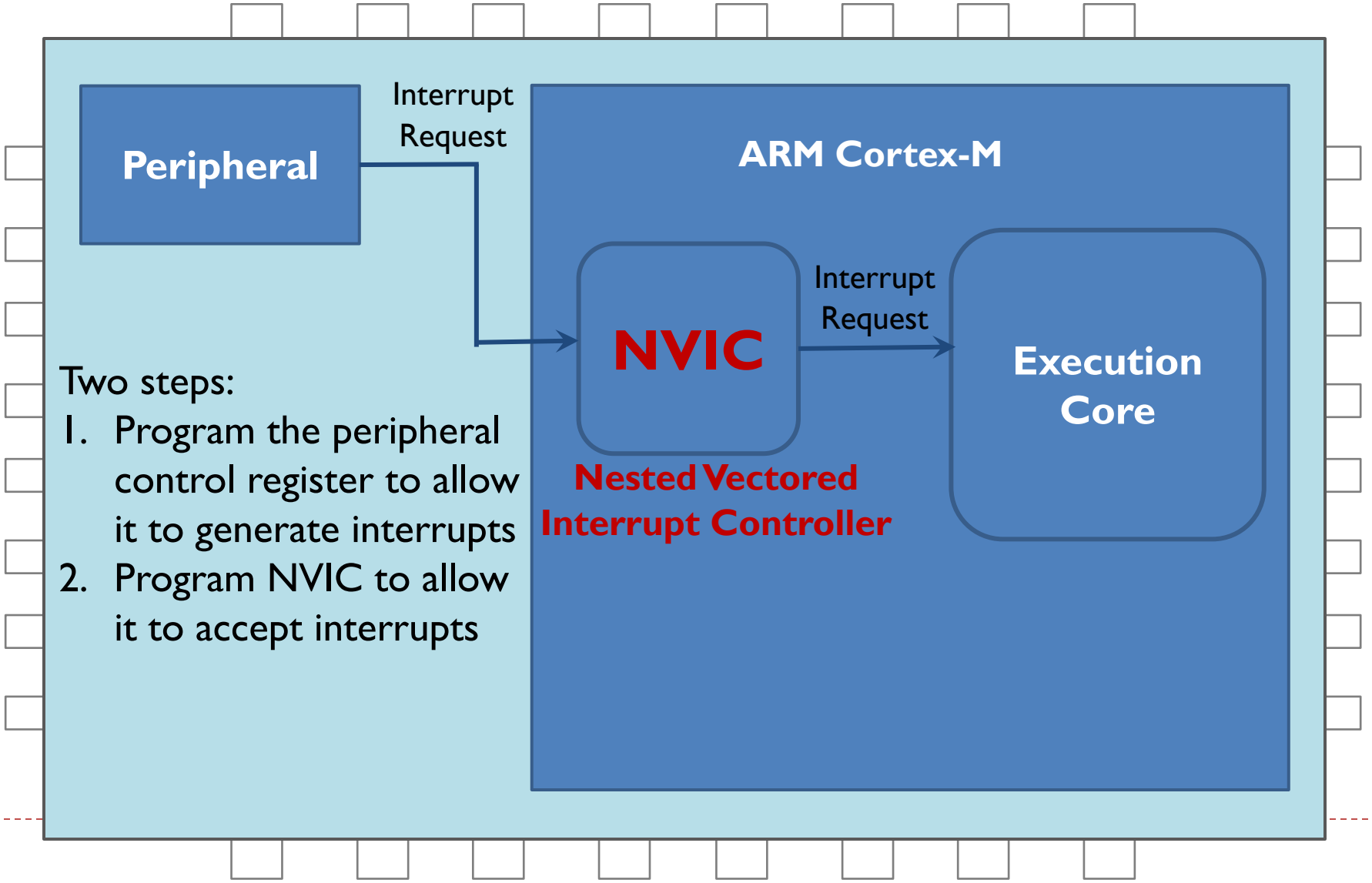
Question: Input Capture



- How does software know a capture event occur?

via interrupt

Recall: Enable an Interrupt



Timer DMA/Interrupt Enable Register (DIER)

1 = enable, 0 = disable

...	8	6	5	4	3	2	1	0
	BIE	TIE	COMIE	CC4IE	CC3IE	CC2IE	**CC1IE**	UIE



Assume: external signal is connected to Timer 4 Channel 1

Enable interrupt

```
TIM4->DIER |= 0x02;          // Enable interrupt for channel 1
NVIC_EnableIRQ(TIM4_IRQn);   // Enable interrupt for timer 4
```

Disable interrupt

```
TIM4->DIER &= ~0x02;          // Disable interrupt for channel 1
NVIC_DisableIRQ(TIM4_IRQn); // Disable interrupt for timer 4
```

Timer DMA/Interrupt Enable Register (DIER)

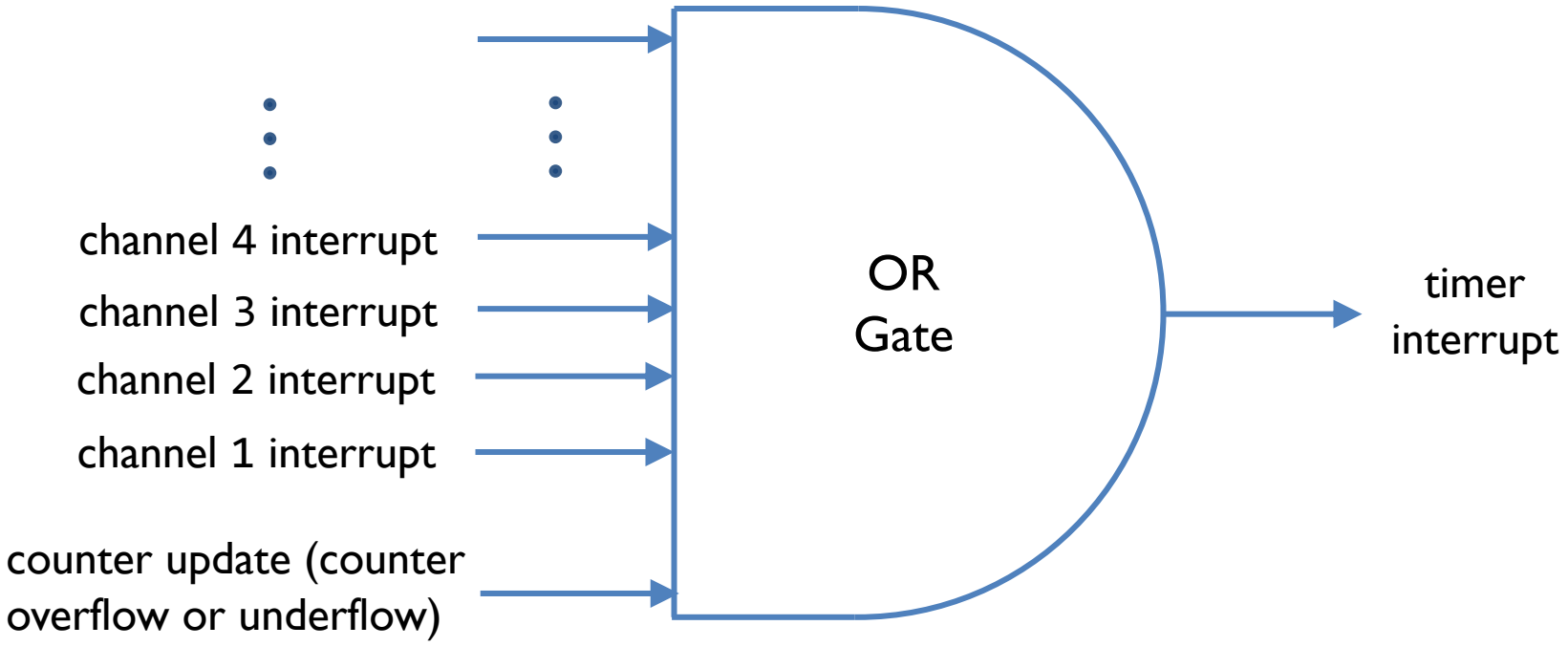
1 = enable, 0 = disable

...	7	6	5	4	3	2	1	0
	...	TIE	COMIE	CC4IE	CC3IE	CC2IE	**CC1IE**	UIE

UIE: Update Interrupt Enable
CC1IE: Capture/compare Channel 1 Interrupt Enable
CC2IE: Capture/compare Channel 2 Interrupt Enable
CC3IE: Capture/compare Channel 3 Interrupt Enable
CC4IE: Capture/compare Channel 4 Interrupt Enable
TIE: Trigger Interrupt Enable
...

Find out the interrupt event

- For a timer, multiple events can trigger an interrupt
- How does the software know which event has trigger the interrupt?



Find out the interrupt event

```
void TIM4_IRQHandler (void) {

  if(TIM4->SR & TIM_SR_UIF) {  // UIF Interrupt

     ...

  } else if (TIM4->SR & TIM_SR_CC1IF) { // Channel #1

     ...

  } else if (TIM4->SR & TIM_SR_CC2IF) { // Channel #2

     ...

  }
  ...

}
```

Find out the interrupt event

```
void TIM4_IRQHandler (void) {

  if(TIM4->SR & TIM_SR_UIF) {  // UIF Interrupt
     TIM4->SR &= ~TIM_SR_UIF;  // Clear interrupt flag
     ...

  } else if (TIM4->SR & TIM_SR_CC1IF) { // Channel #1
     TIM4->SR &= ~ TIM_SR_CC1IF;        // Clear interrupt flag
     ...

  } else if (TIM4->SR & TIM_SR_CC2IF) { // Channel #2
     TIM4->SR &= ~ TIM_SR_CC2IF;        // Clear interrupt flag
     ...

  }
  ...

}
```

Hardware sets these flags, and software needs to clear them.

Timer Status Register (SR)

TIMx_SR = 0 (default)

...	8	6	5	4	3	2	1	0
	BIF	TIF	COMIF	CC4IF	CC3IF	CC2IF	CC1IF	**UIF**

UIF is set by hardware and needs to be cleared by software.

```
if (TIM4->SR & 0x01 == 0x01) { // Test UIF

    // If hardware sets UIF, do some actions
    ...

    // Clear UIF
    TIM4->SR &= ~0x01;
}
```

Timer Status Register (SR)

`TIMx_SR = 0 (default)`

...	8	6	5	4	3	2	1	0
	BIF	TIF	COMIF	CC4IF	CC3IF	CC2IF	**CC1IF**	UIF

CC1IF: Capture/Compare 1 interrupt flag

If channel CC1 is configured as output, this flag is set by hardware when the counter matches the compare value, with some exception in center-aligned mode. It is cleared by software.

- 0: No match.
- 1: The content of the counter TIMx_CNT matches the content of the TIMx_CCR1 register.

When the contents of TIMx_CCR1 are greater than the contents of TIMx_ARR, the CC1IF bit goes high on the counter overflow (in upcounting and up/down-counting modes) or underflow (in downcounting mode)

Timer Status Register (SR)

TIMx_SR = 0 (default)

...	8	6	5	4	3	2	1	0
	BIF	TIF	COMIF	CC4IF	CC3IF	CC2IF	**CC1IF**	UIF

CC1IF: Capture/Compare 1 interrupt flag

If channel CC1 is configured as input, this bit is set by hardware on a capture. It is cleared by software or by reading the TIMx_CCR1 register.

- 0: No input capture occurred
- 1: The counter value has been captured in TIMx_CCR1 register

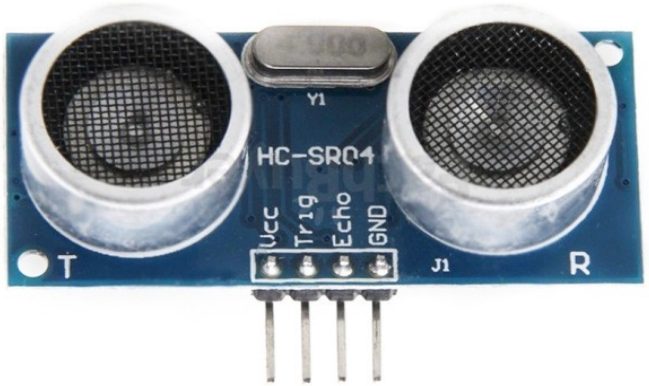
Ultrasonic Distance Sensor

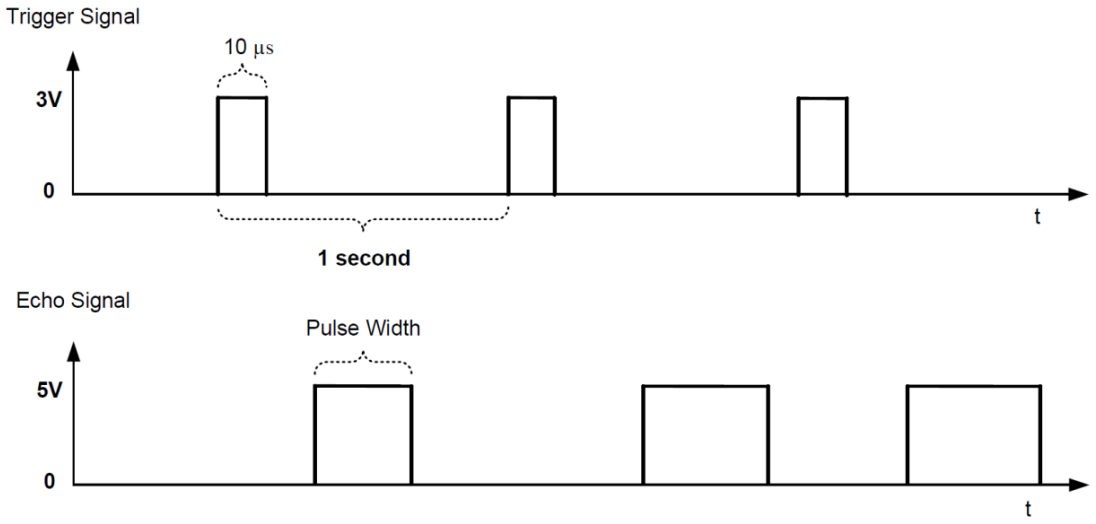


$$Distance = \frac{Round\ Trip\ Time \times Speed\ of\ Sound}{2}$$

$$= \frac{Round\ Trip\ Time(\mu s)}{58}$$

Ultrasonic Distance Sensor





The echo pulse width corresponds to round-trip time.

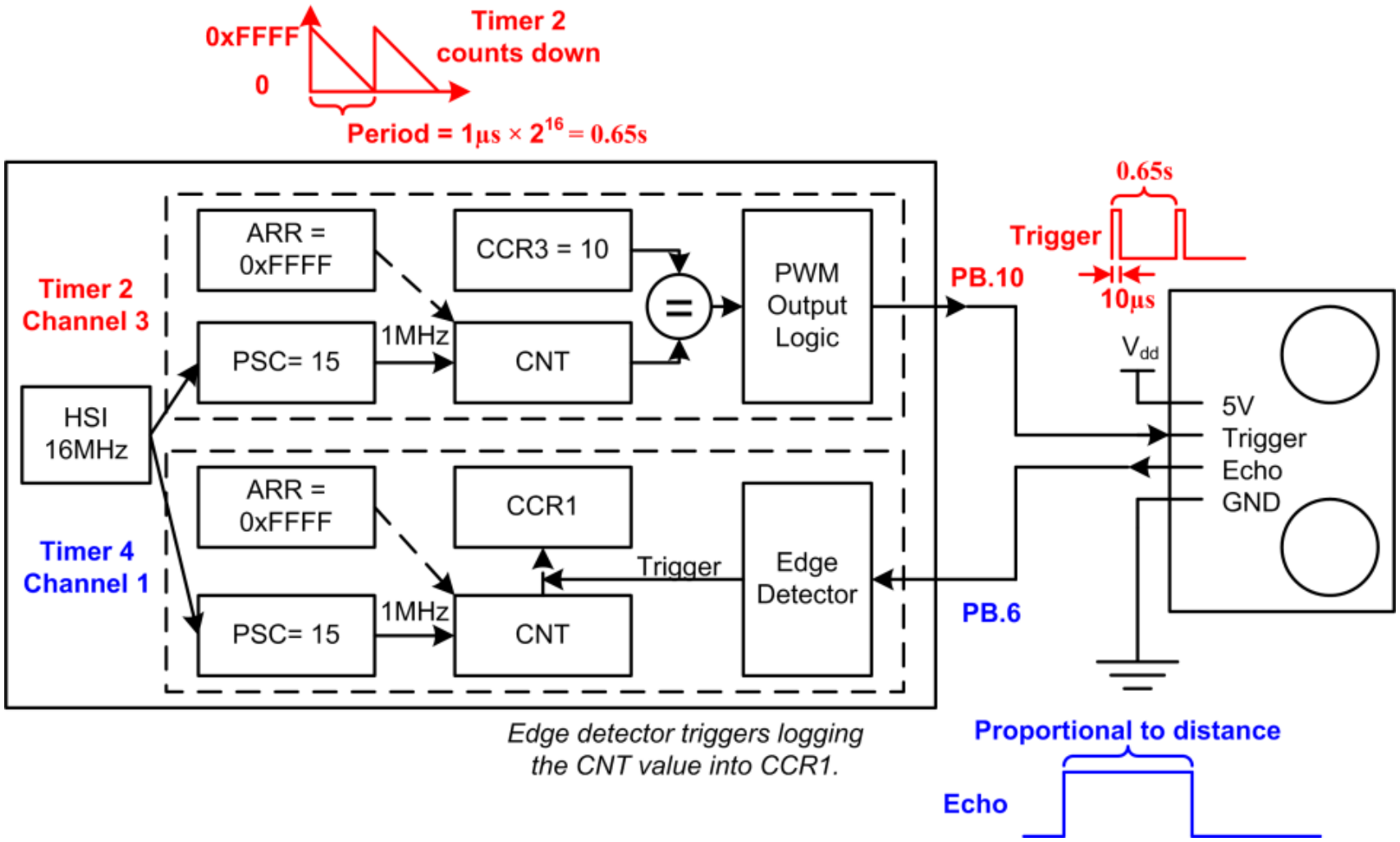
$$Distance\ (cm) = \frac{Pulse\ Width\ (\mu s)}{58}$$

or

$$Distance\ (inch) = \frac{Pulse\ Width\ (\mu s)}{148}$$

- The sensor can measure a distance between 2 cm and 400 cm.
- If the echo pulse width is 38*ms*, no obstacle is detected.

Ultrasonic Distance Sensor



Edge detector triggers logging the CNT value into CCR1.